



**Kotlin
Munich**
User Group

From Zero to Kotlin GPT

in 45 Minutes

Michal Harakal

Deutsche Telekom AG, 2024





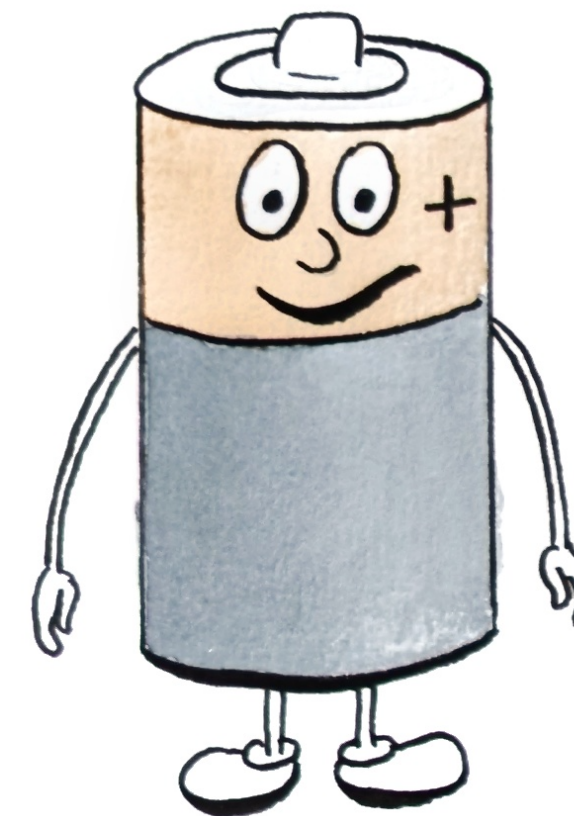
**Kotlin
Munich**
User Group

From Zero to Kotlin GPT

in 45 Minutes

Michal Harakal

Deutsche Telekom AG, 2024

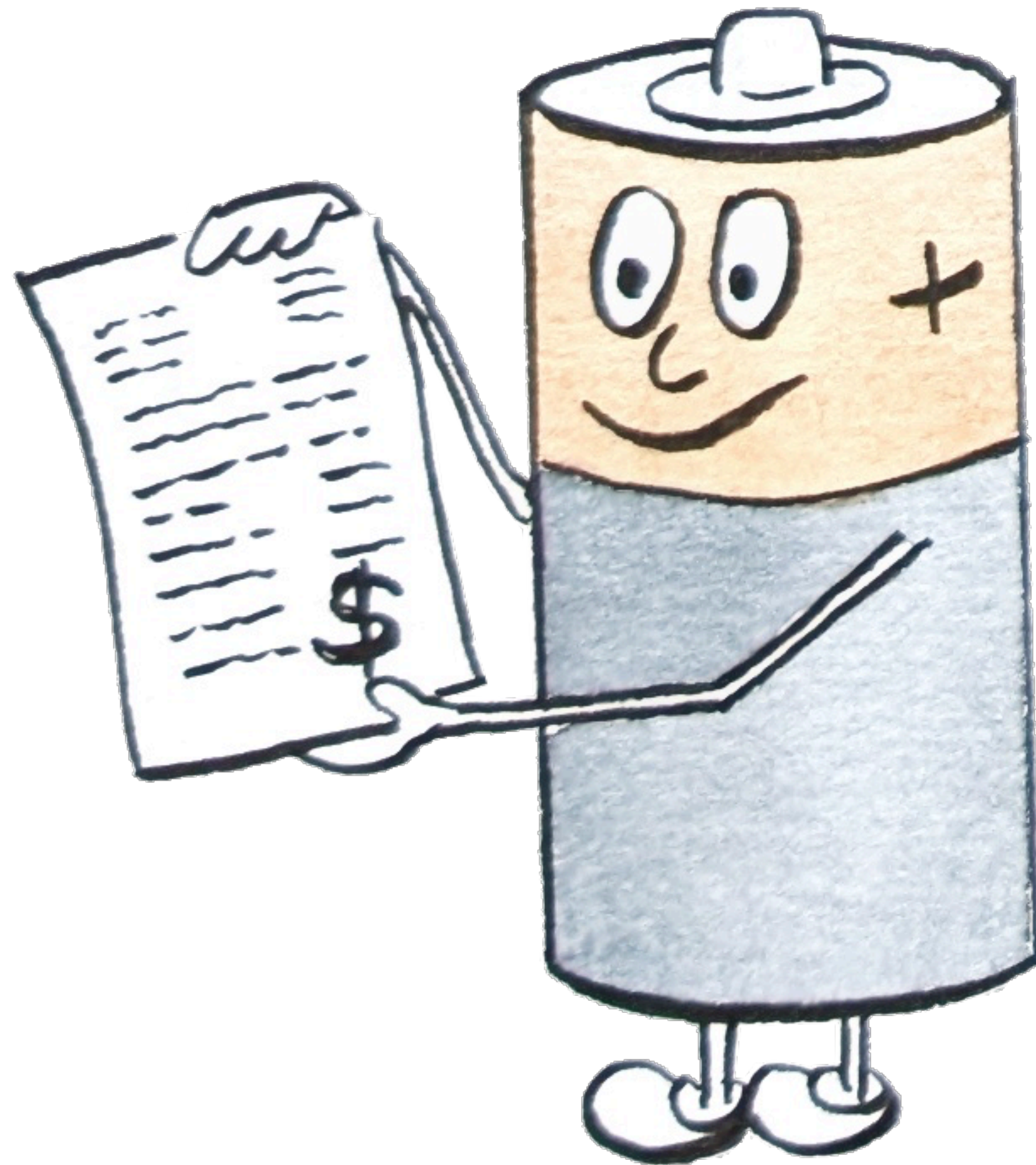


(C) Adriana Harakalova, 2024



Let's talk about ChatGPT & Co first

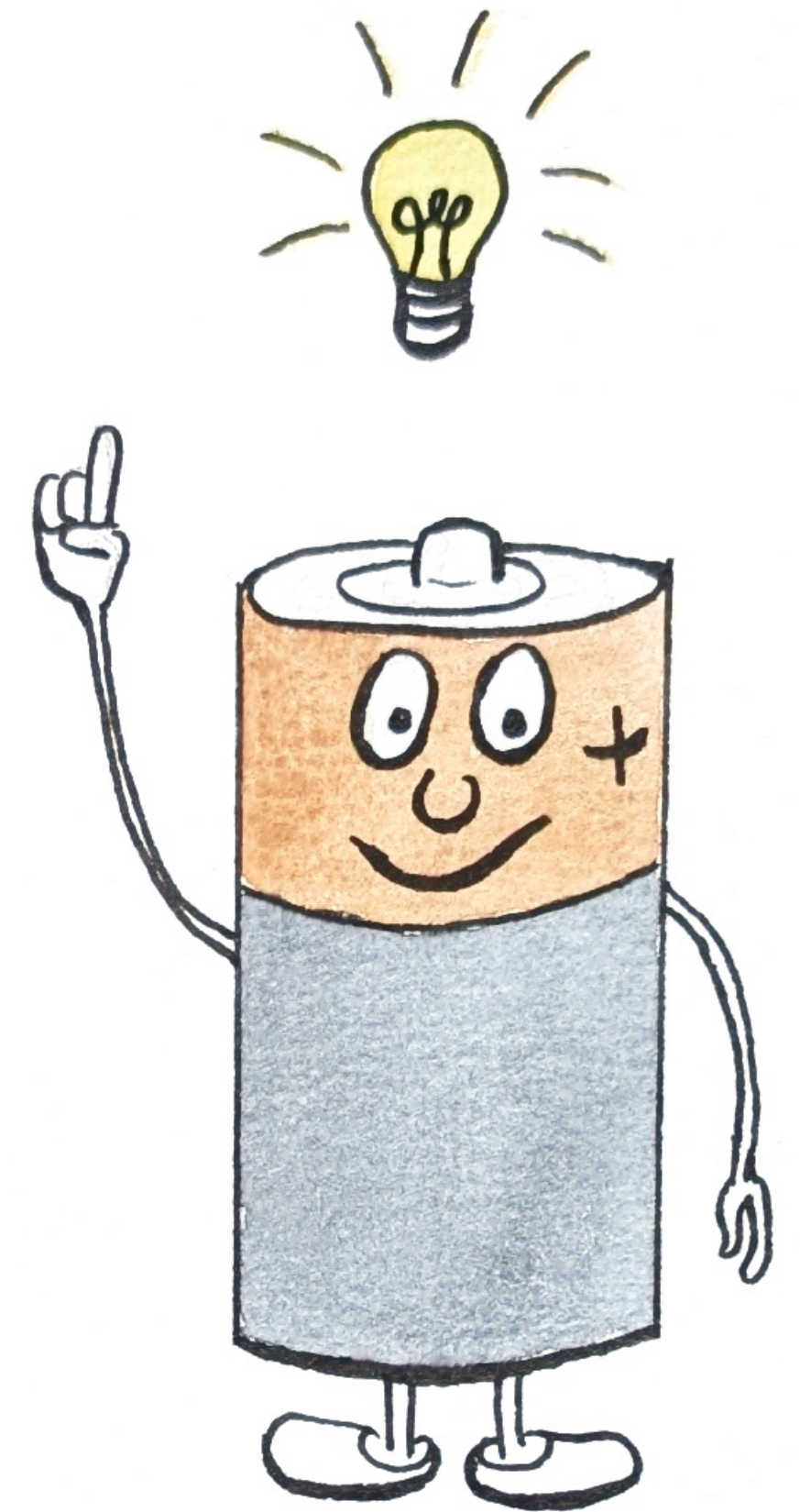
Let's talk about ChatGPT & Co first



(C) Adriana Harakalova, 2024

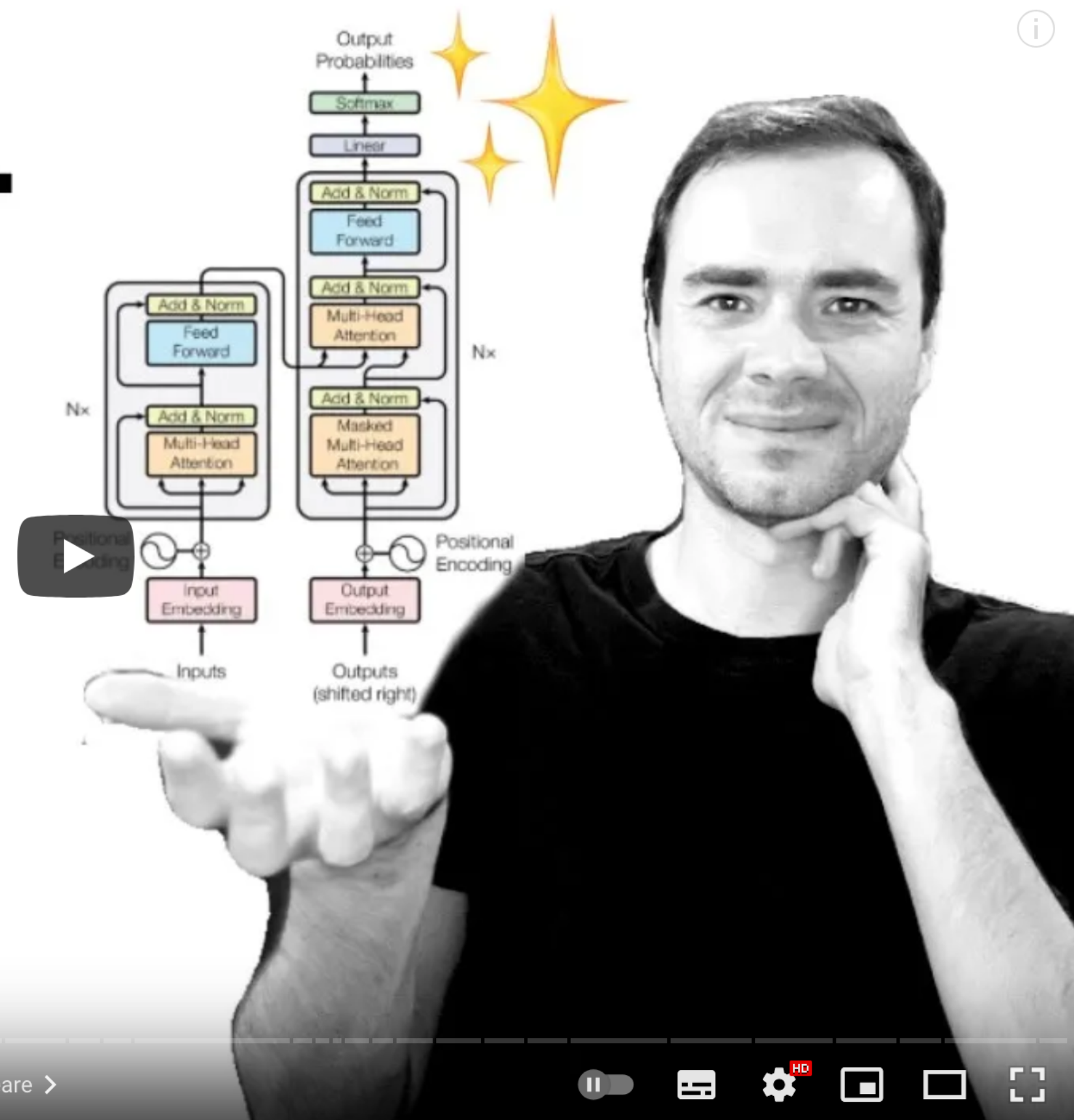
**What I cannot create,
I do not understand.
Know how to solve every problem that
has been solved.**

Richard Feynman, 1988

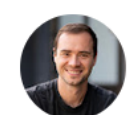


(C) Adriana Harakalova, 2024

LET'S BUILD GPT. FROM SCRATCH. IN CODE. SPELLED OUT.



Let's build GPT: from scratch, in code, spelled out.



Andrej Karpathy
540.000 Abonnenten

Abonnieren

107.486



Teilen

Speichern



<https://www.youtube.com/watch?v=kCc8FmEb1nY>

Michal Harakal, 2024/11

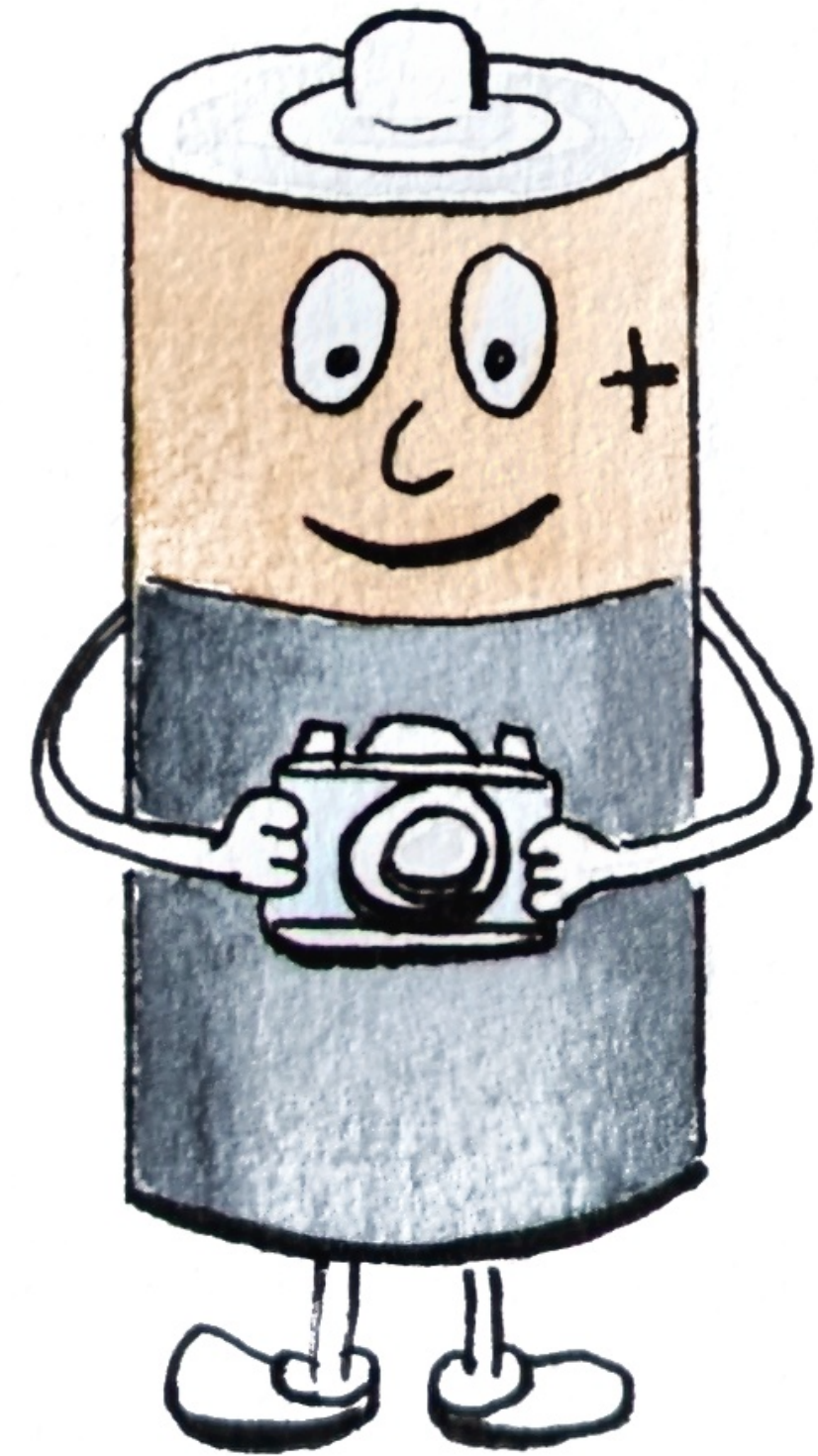
Let's build GPT: from scratch, in code, spelled out.

Andrej Karpathy

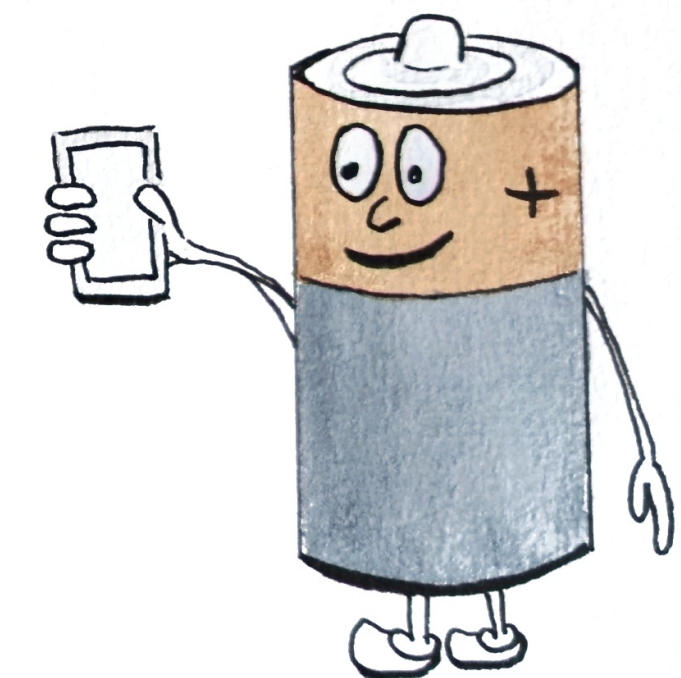
- Simplified GTP-2 model with approx. 10 million parameters
- 250 lines of Python code
- uses PyTorch
- decoder only



Kotlin

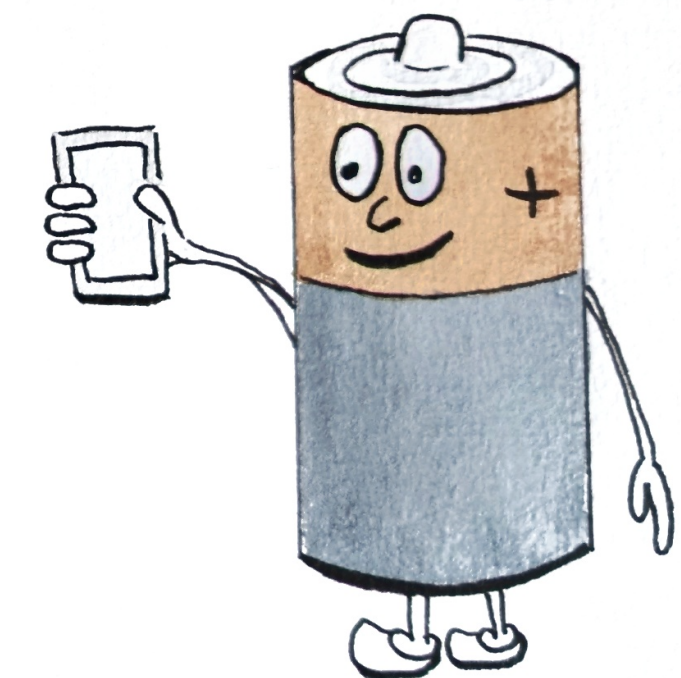


(C) Adriana Harakalova, 2024



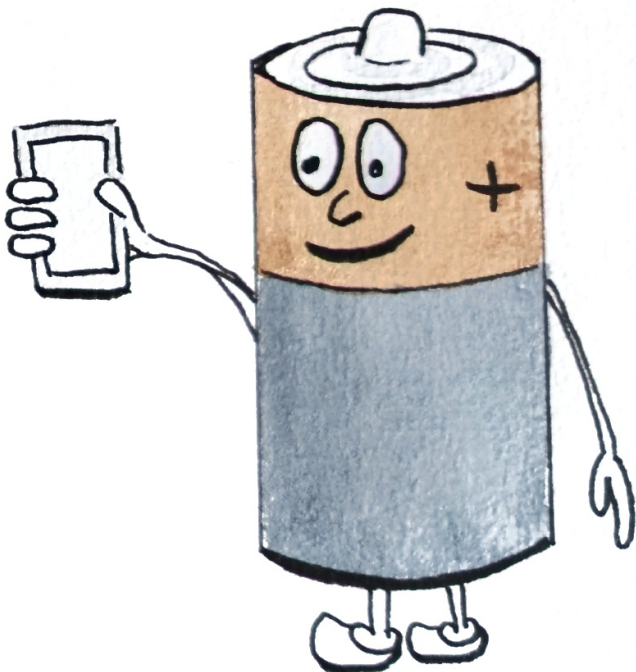
(C) Adriana Harakalova, 2024





(C) Adriana Harakalova, 2024





(C) Adriana Harakalova, 2024



+



=



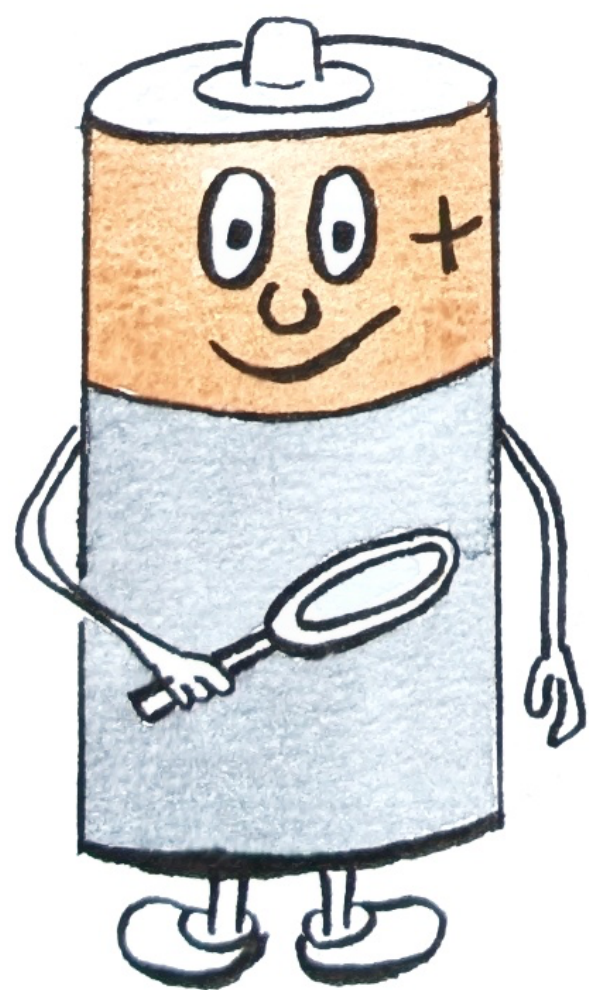
Kotlin Multiplatform

Source code sharing

- Cross-platform development
- Support for native performance - Kotlin Native
- Cross-platform UI with Compose Multiplatform



Generative Pre-trained Transformers



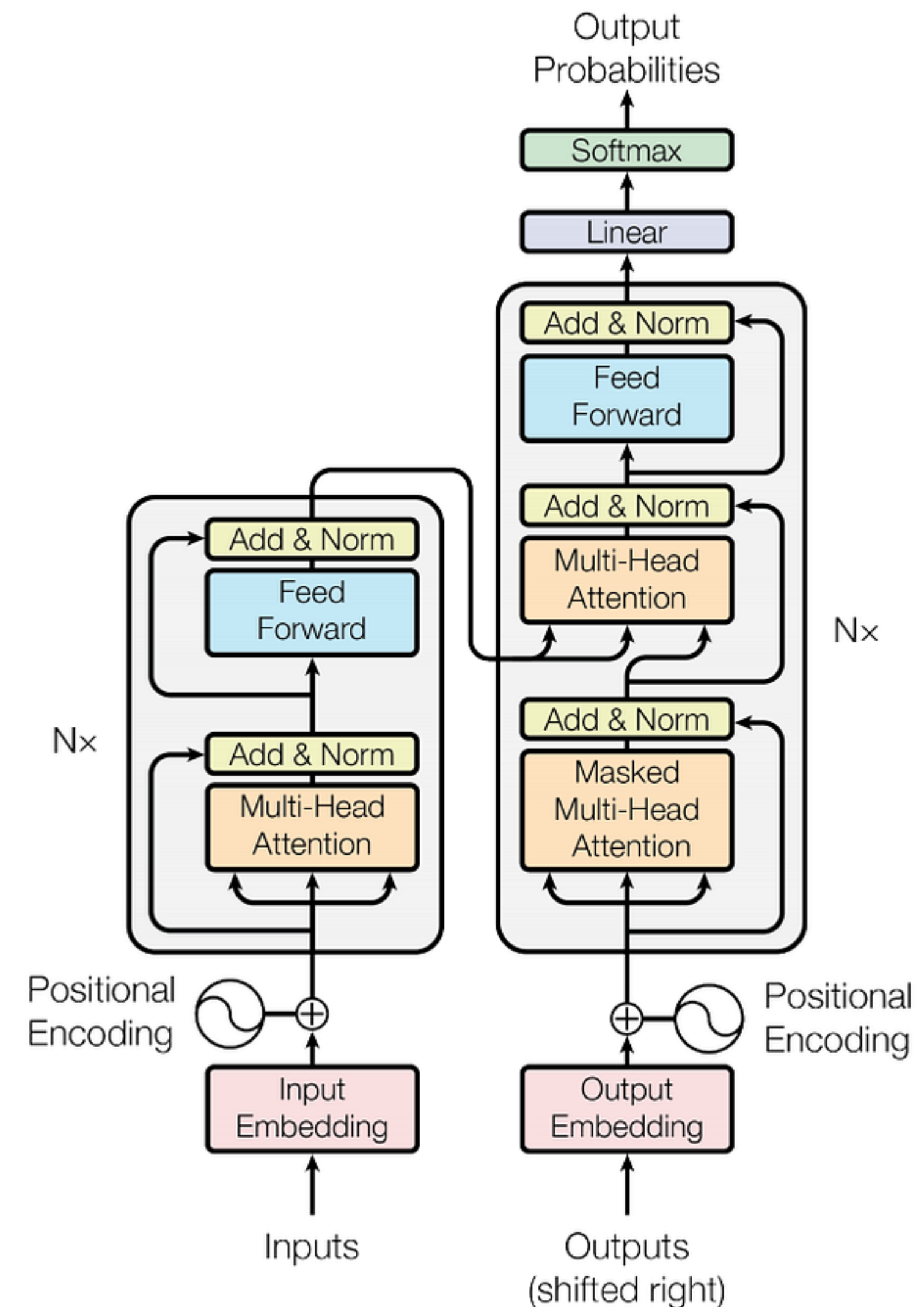
(C) Adriana Harakalova, 2024

Transformers

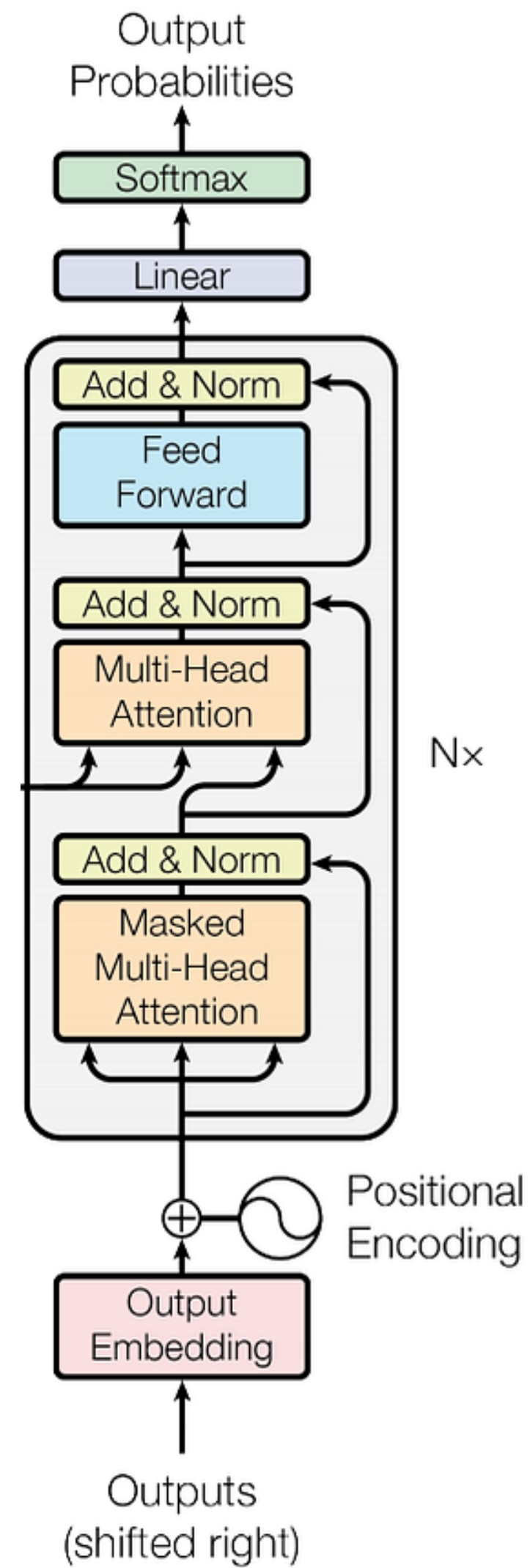
Attention Is All You Need

- Published 2017
- Google Brain, Google Research et al.
- Transformer model architecture
- Efficient computation as goal
- Translations

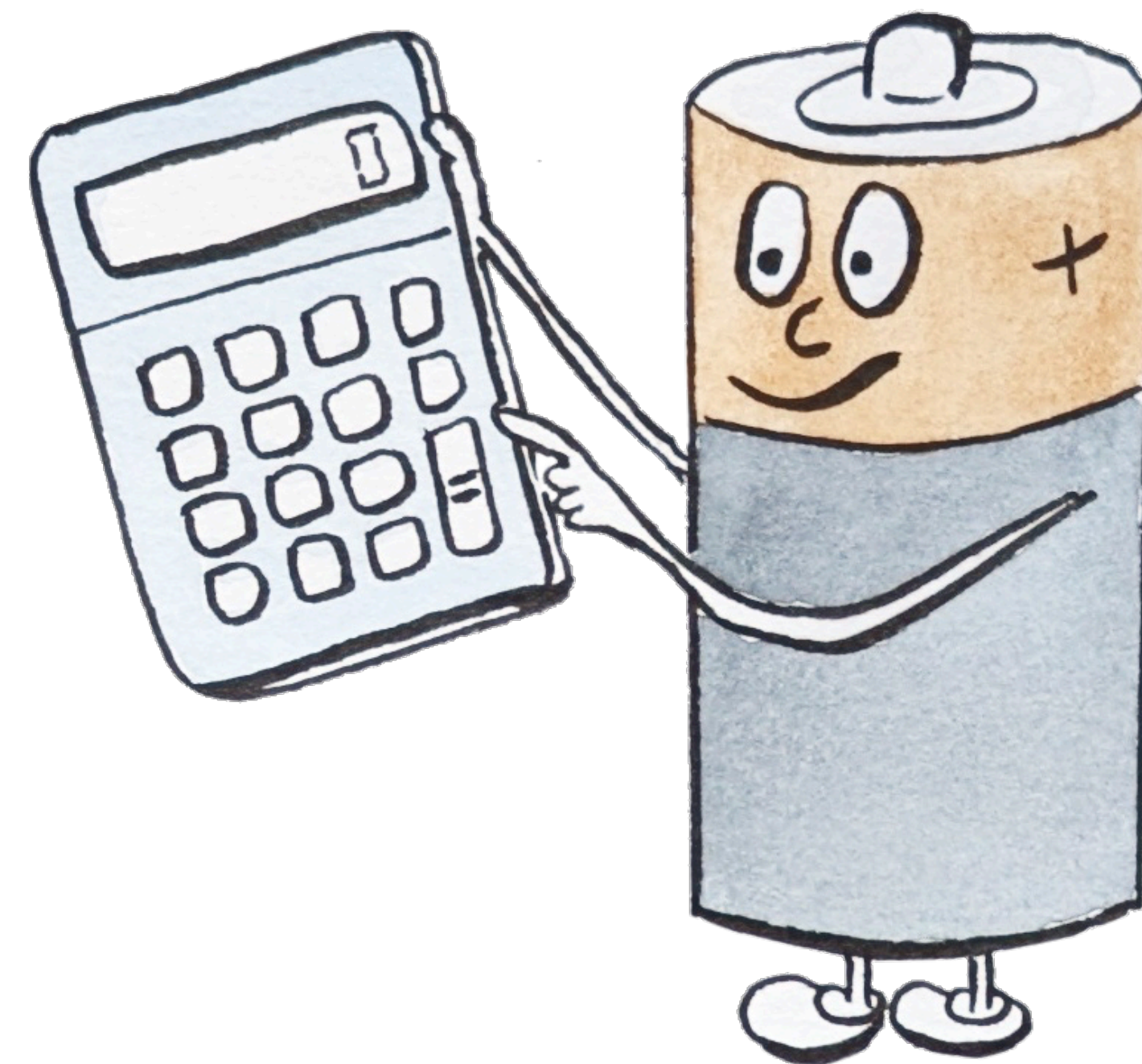
Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin



Decoder only

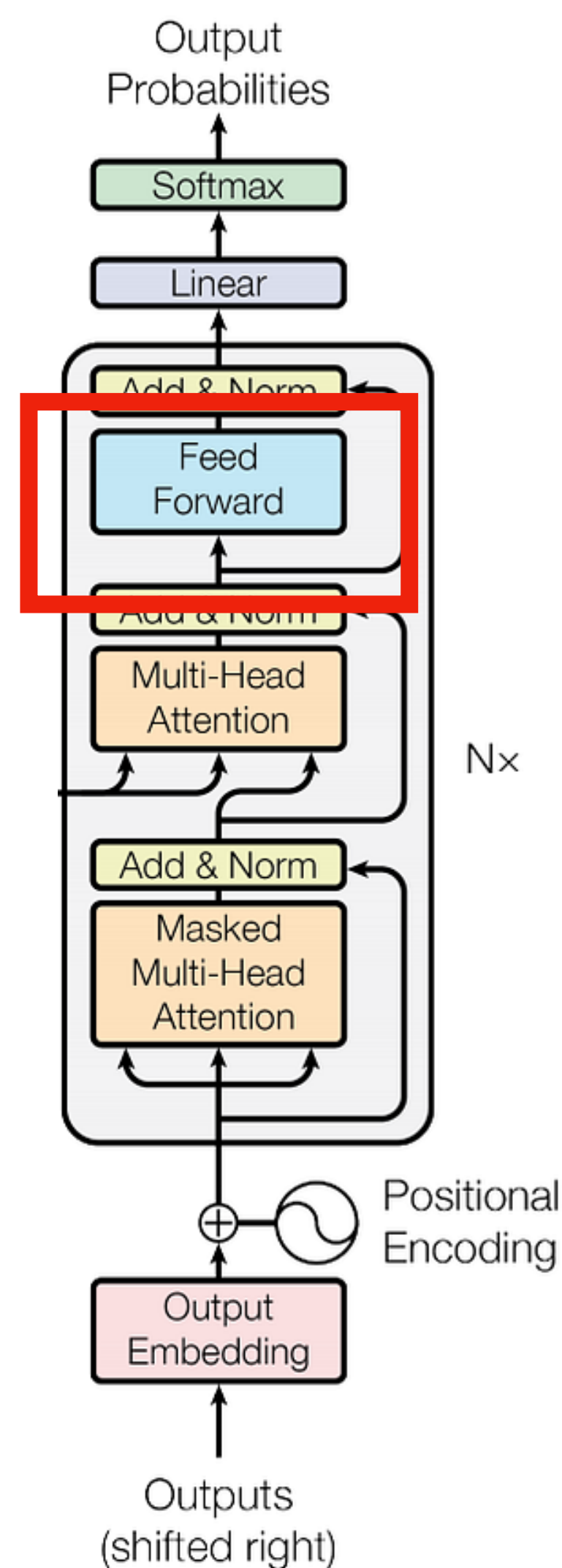


Pre-trained

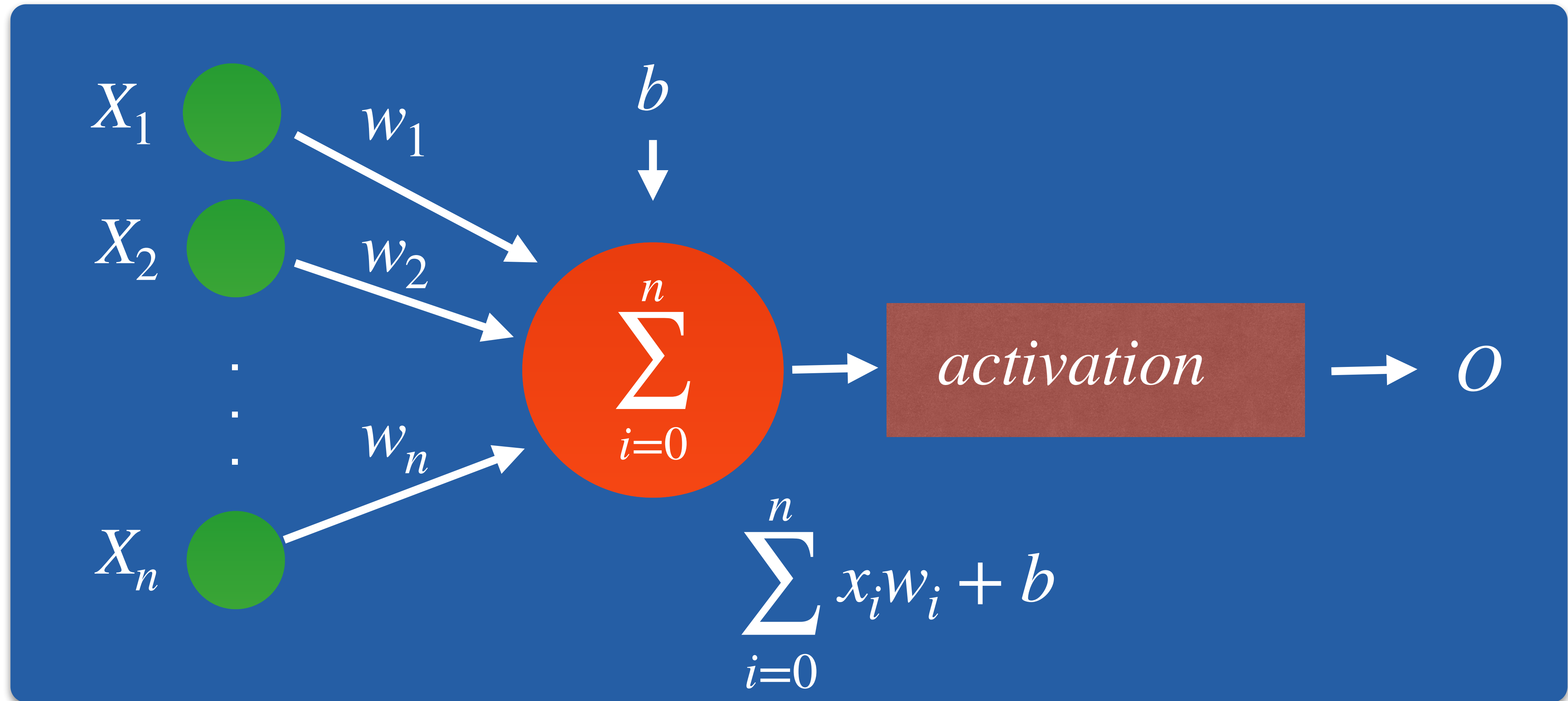


(C) Adriana Harakalova, 2024

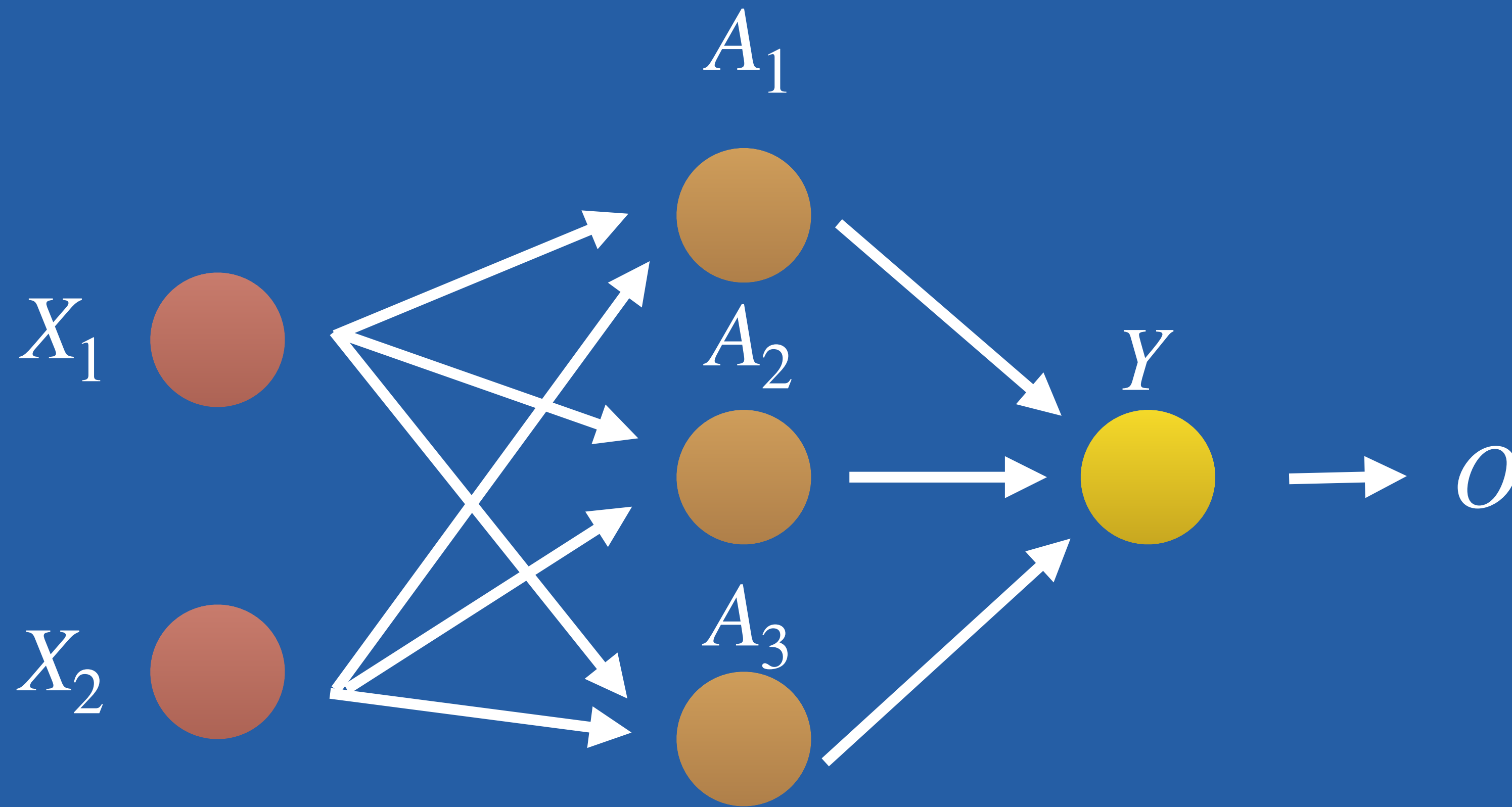
Deep neural network



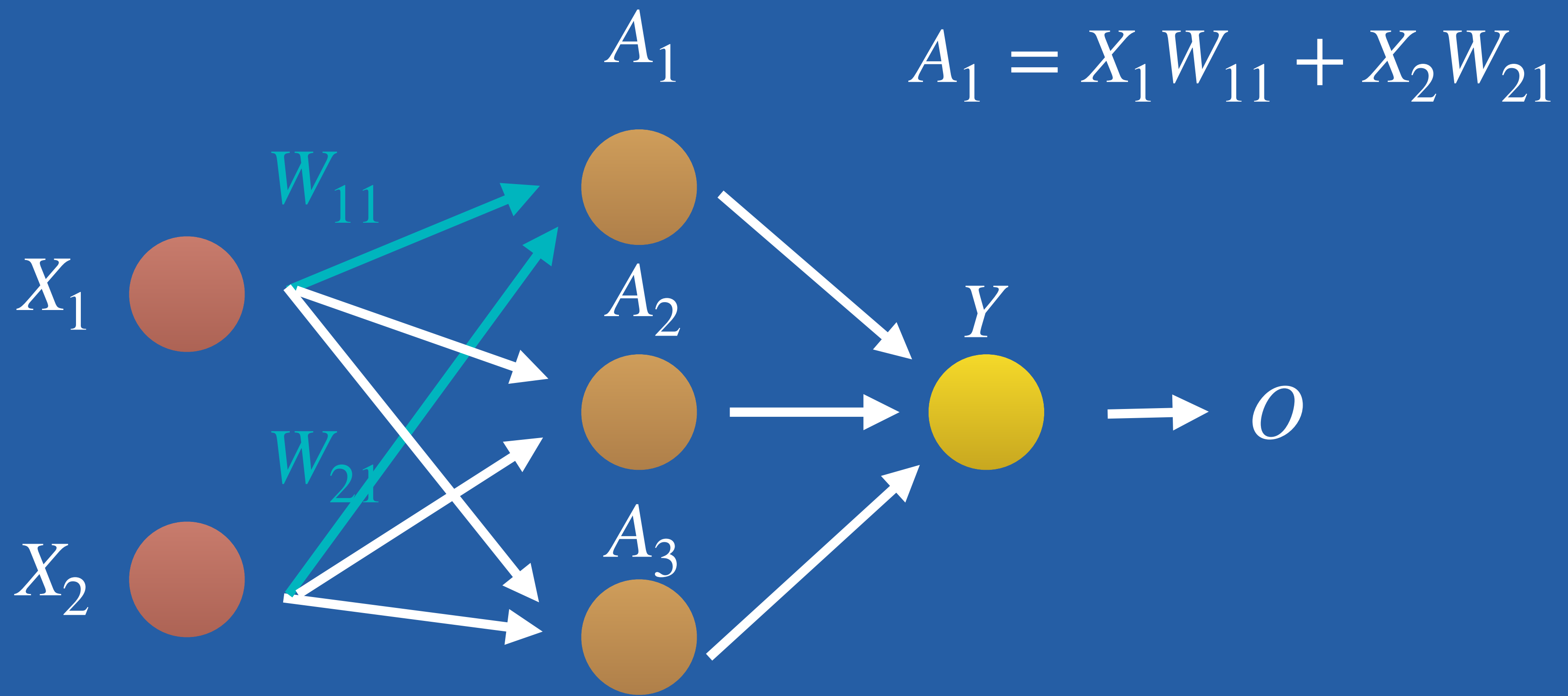
Neuron



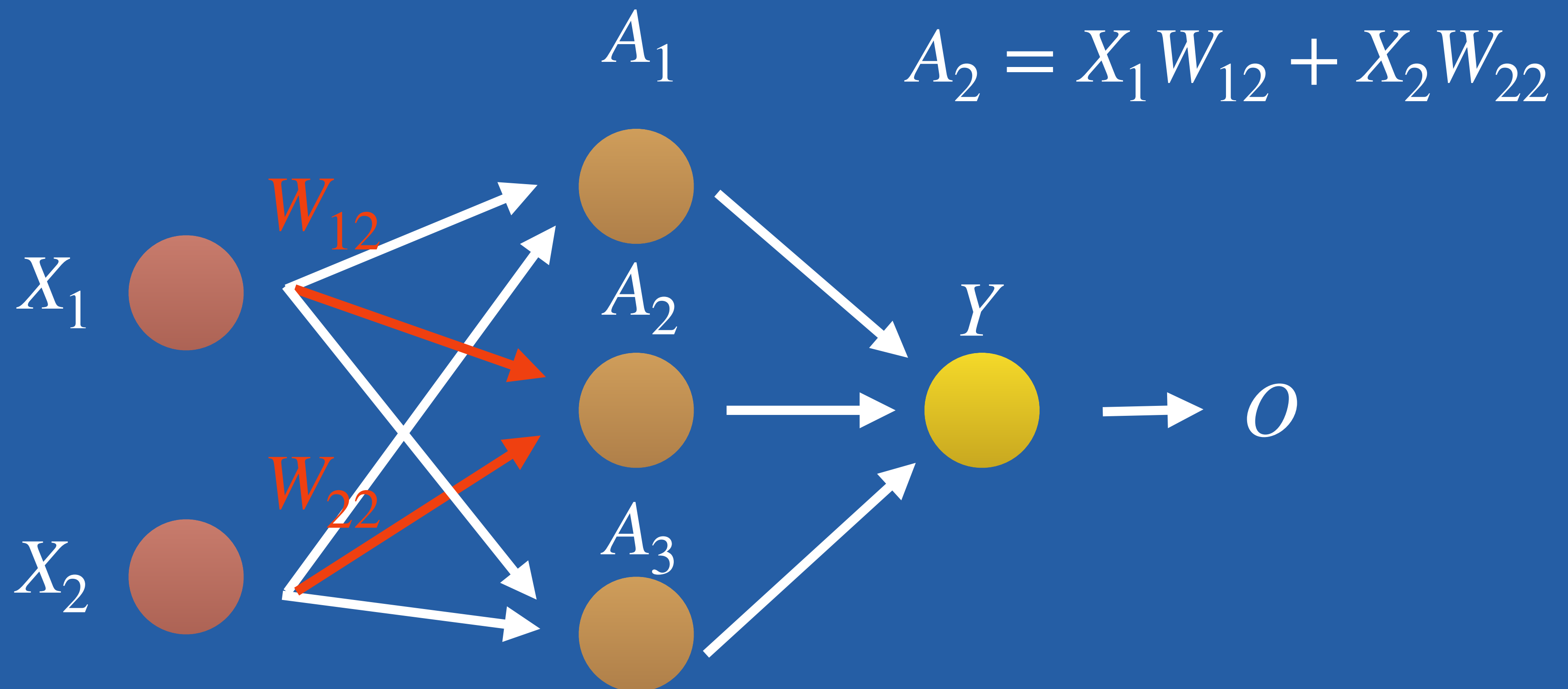
MLP - Multilayer Perceptron



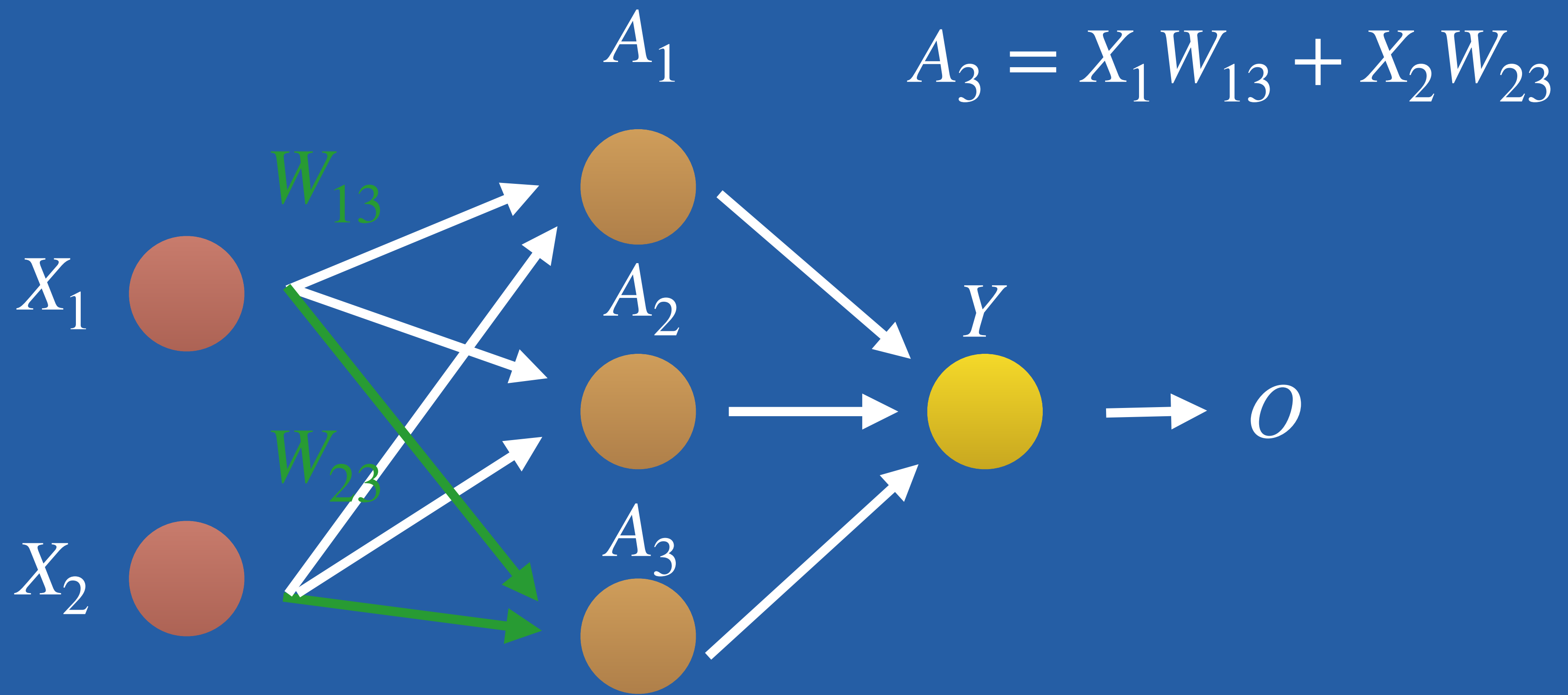
Forward propagation



Forward propagation



Forward propagation



Matrix Multiplication

$$A = X \cdot W$$

$$A_{11} = X_1 W_{11} + X_2 W_{21}$$

$$A_{12} = X_1 W_{12} + X_2 W_{22}$$

$$A_{13} = X_1 W_{13} + X_2 W_{23}$$

$$X = \begin{bmatrix} X_1 & X_2 \end{bmatrix}$$

$$W = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix}$$

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} \end{bmatrix}$$

$$A = X \cdot W$$

Matrices multiplication

In Kotlin with lists of doubles

$$A = X \cdot W$$

```
// Define a typealias for better readability
typealias Matrix = List<List<Double>>

class Matrix(private val data: List<List<Double>>) {
    private val numRows = data.size
    private val numCols = data.firstOrNull()?.size ?: 0

    fun matmul(other: Matrix): Matrix {
        require(numCols == other.numRows) {
            "Number of columns of A ($numCols) must equal number of rows of B (${other.numRows})."
        }

        val resultData = List(numRows) { i ->
            List(other.numCols) { j ->
                (0 until numCols).sumOf { k ->
                    data[i][k] * other.data[k][j]
                }
            }
        }
        return Matrix(resultData)
    }
}
```

ReLU

Activation function

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

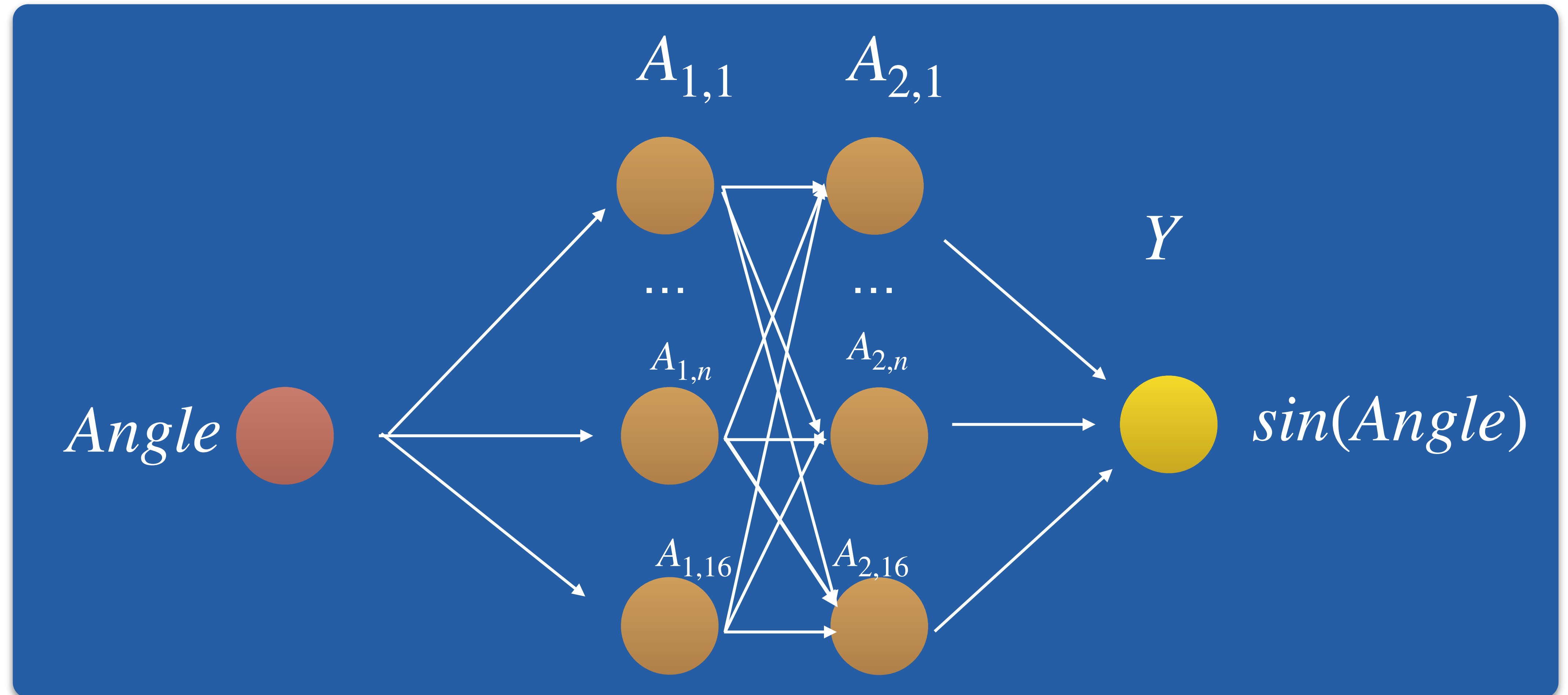
```
// ReLU Activation Function
fun Matrix.relu(): Matrix {
    val reluData = data.map { row ->
        row.map { value ->
            max(0.0, value)
        }
    }
    return Matrix(reluData)
}
```

Forward Propagation in Kotlin

Hidden layer

```
class Linear() {  
    val weights: Tensor  
    val bias: Tensor  
  
    fun forward(input: Tensor): Tensor {  
        val output = input.matmul(weight.value.t()) + bias.value  
        return output  
    }  
}
```

Sinus calculation with neuronal network



Forward Propagation in Kotlin

MLP

```
class SineMLP : Module() {  
    private val inputLayer: Module = Linear(1, 16)  
    private val hiddenLayer1: Module = Linear(16, 16)  
    private val hiddenLayer2: Module = Linear(16, 16)  
    private val outputLayer: Module = Linear(16, 1)  
    private val relu: Module = ReLU()  
  
    override fun forward(input: Tensor): Tensor {  
        var x = input  
        x = inputLayer.forward(x)  
        x = relu.forward(x)  
        x = hiddenLayer1.forward(x)  
        x = relu.forward(x)  
        x = hiddenLayer2.forward(x)  
        x = relu.forward(x)  
        x = outputLayer.forward(x)  
        return x  
    }  
}
```

Forward Propagation in Kotlin

MLP with DSL

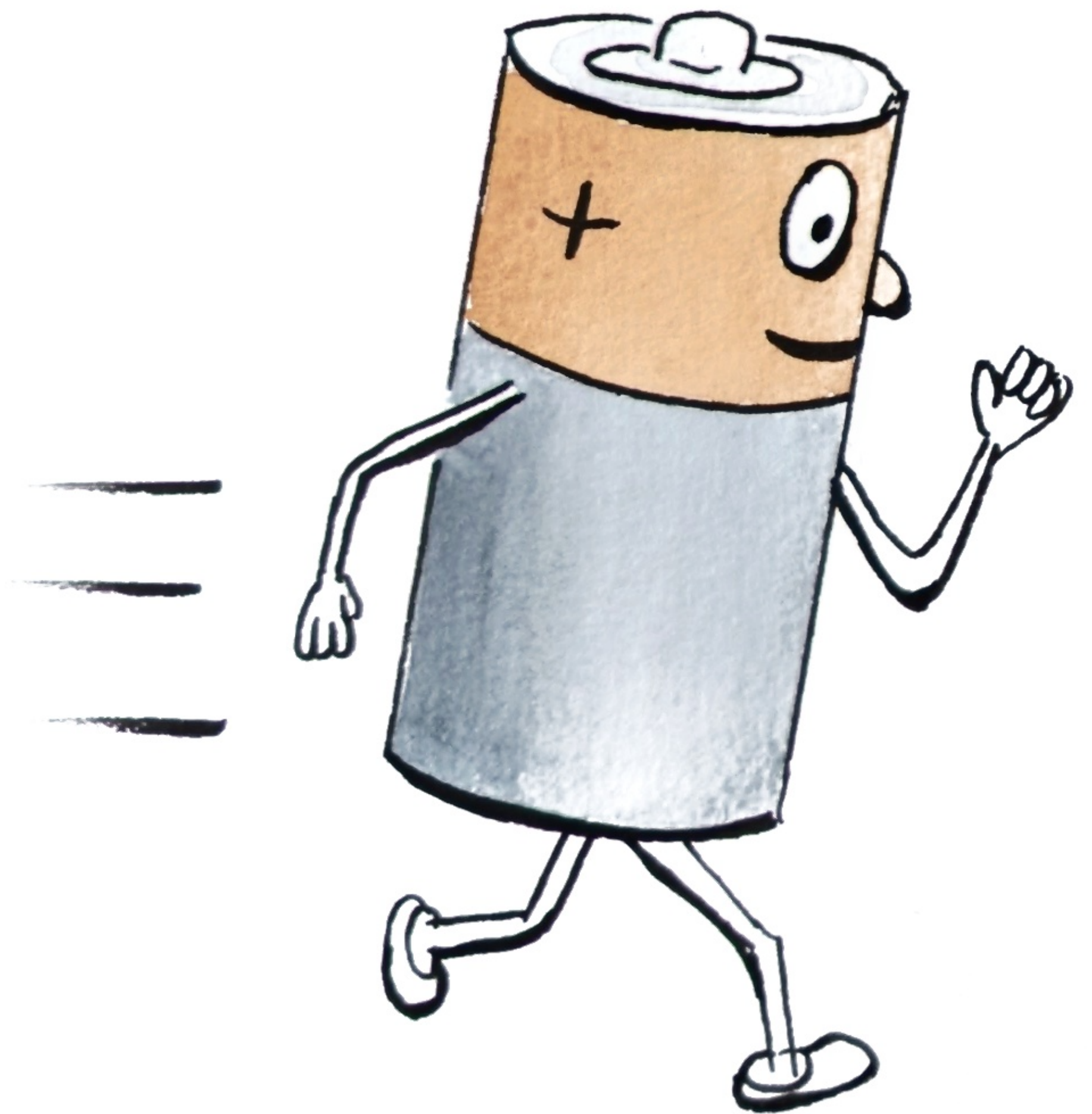
```
class SineNN(override val name: String="sin") : Module() {  
    private val sineModule = sequential{  
        input(1)  
        linear(16, "layer1") {  
            activation = relu  
        }  
        linear(16, "layer2") {  
            activation = relu  
        }  
        linear(1, "output_layer")  
    }  
    override fun forward(input: Tensor): Tensor =  
        sineModule.forward(input)  
}
```

Forward Propagation in Kotlin

1 input, 2 hidden layers with 16 neurons, 1 output

	Weight	Biases	Sum
Input	16	16	32
1.layer => 2.layer	256	16	272
2.layer => output	16	1	17
Sum	288	33	321

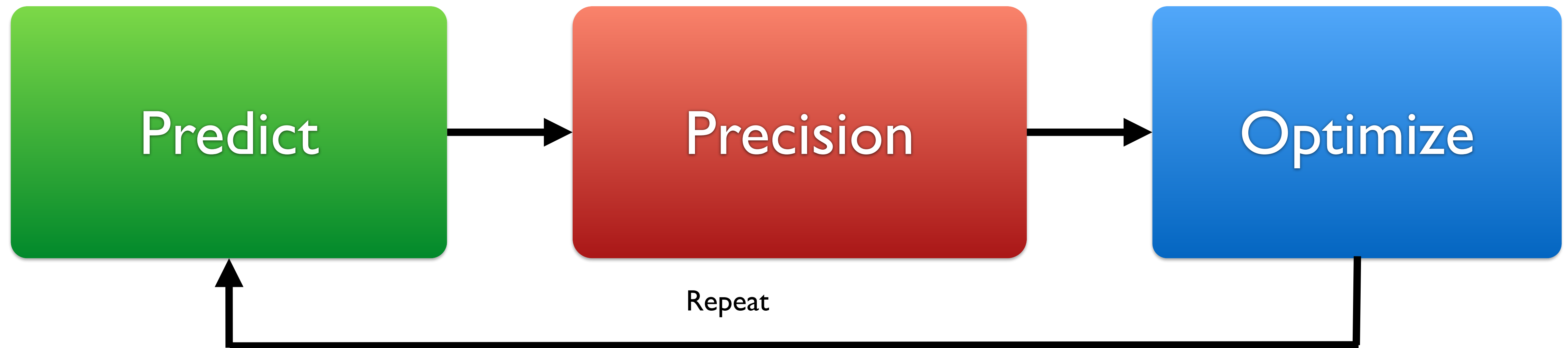
Trained



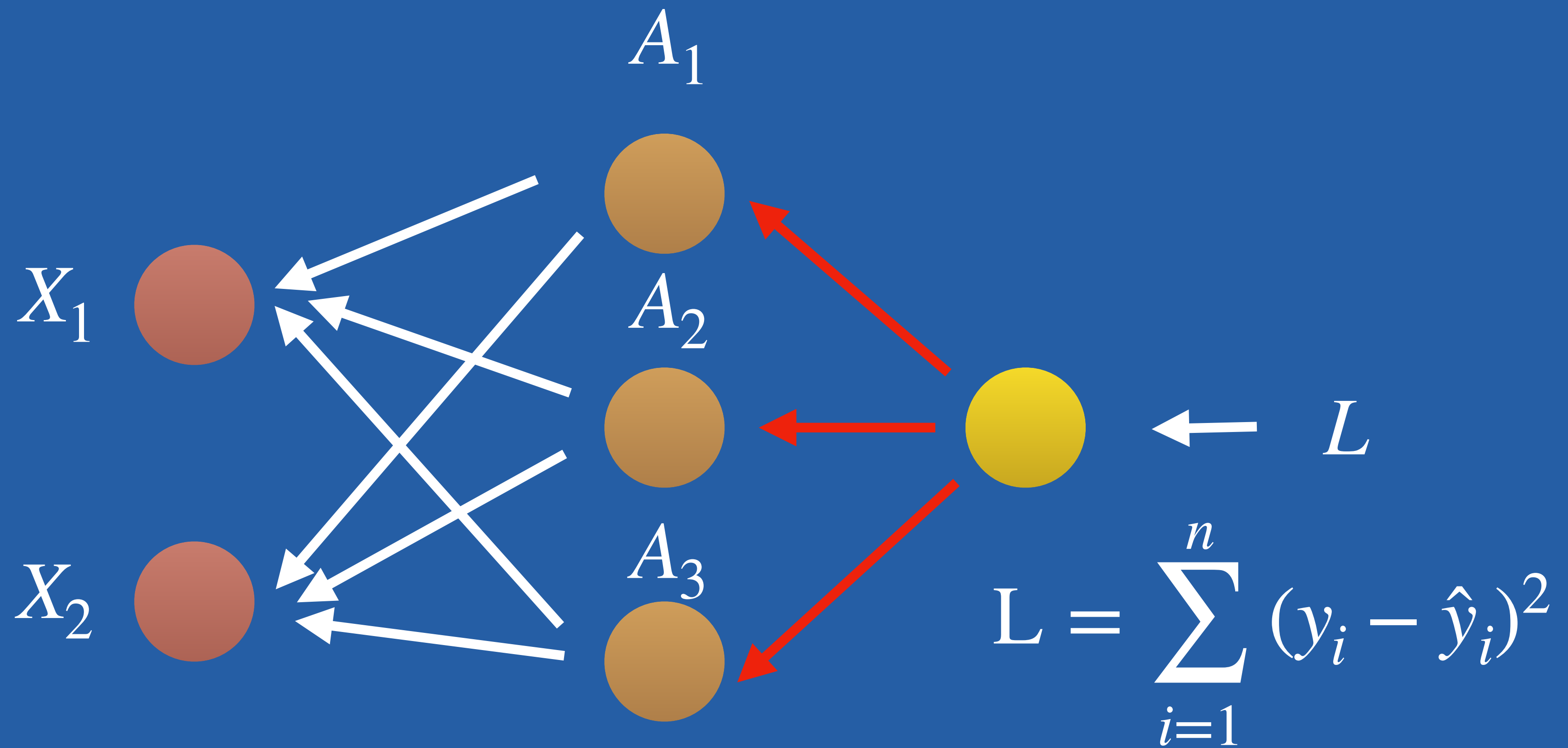
(C) Adriana Harakalova, 2024

Supervised learning

PARADIGM OF MACHINE LEARNING



Backpropagation



Loss function

Mean squared error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2$$

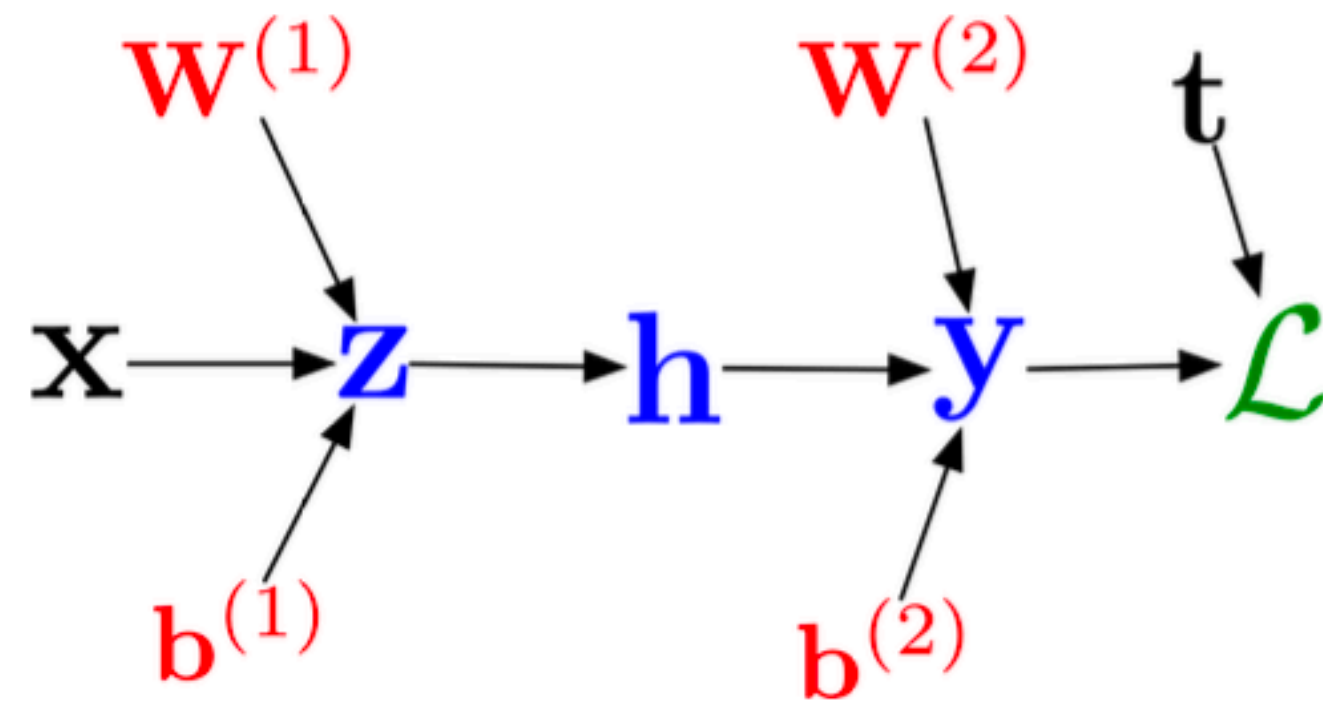
```
// Extension function for MSE calculation
fun Tensor.meanSquaredError(other: Tensor): Double {
    // Check if the tensors have the same size
    if (this.size != other.size) {
        throw IllegalArgumentException("Tensors must have the same size.")
    }

    // Calculate the squared errors and the mean
    var sum = 0.0
    for (i in 0 until this.size) {
        val error = this[i] - other[i]
        sum += error * error
    }
    return sum / this.size
}
```

Deep neural networks

Math

MLP example in vectorized form:



Forward pass:

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{h} = \sigma(\mathbf{z})$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \|\mathbf{t} - \mathbf{y}\|^2$$

Backward pass:

$$\bar{\mathcal{L}} = 1$$

$$\bar{\mathbf{y}} = \bar{\mathcal{L}}(\mathbf{y} - \mathbf{t})$$

$$\overline{\mathbf{W}^{(2)}} = \bar{\mathbf{y}}\mathbf{h}^\top$$

$$\overline{\mathbf{b}^{(2)}} = \bar{\mathbf{y}}$$

$$\bar{\mathbf{h}} = \mathbf{W}^{(2)\top} \bar{\mathbf{y}}$$

$$\bar{\mathbf{z}} = \bar{\mathbf{h}} \circ \sigma'(\mathbf{z})$$

$$\overline{\mathbf{W}^{(1)}} = \bar{\mathbf{z}}\mathbf{x}^\top$$

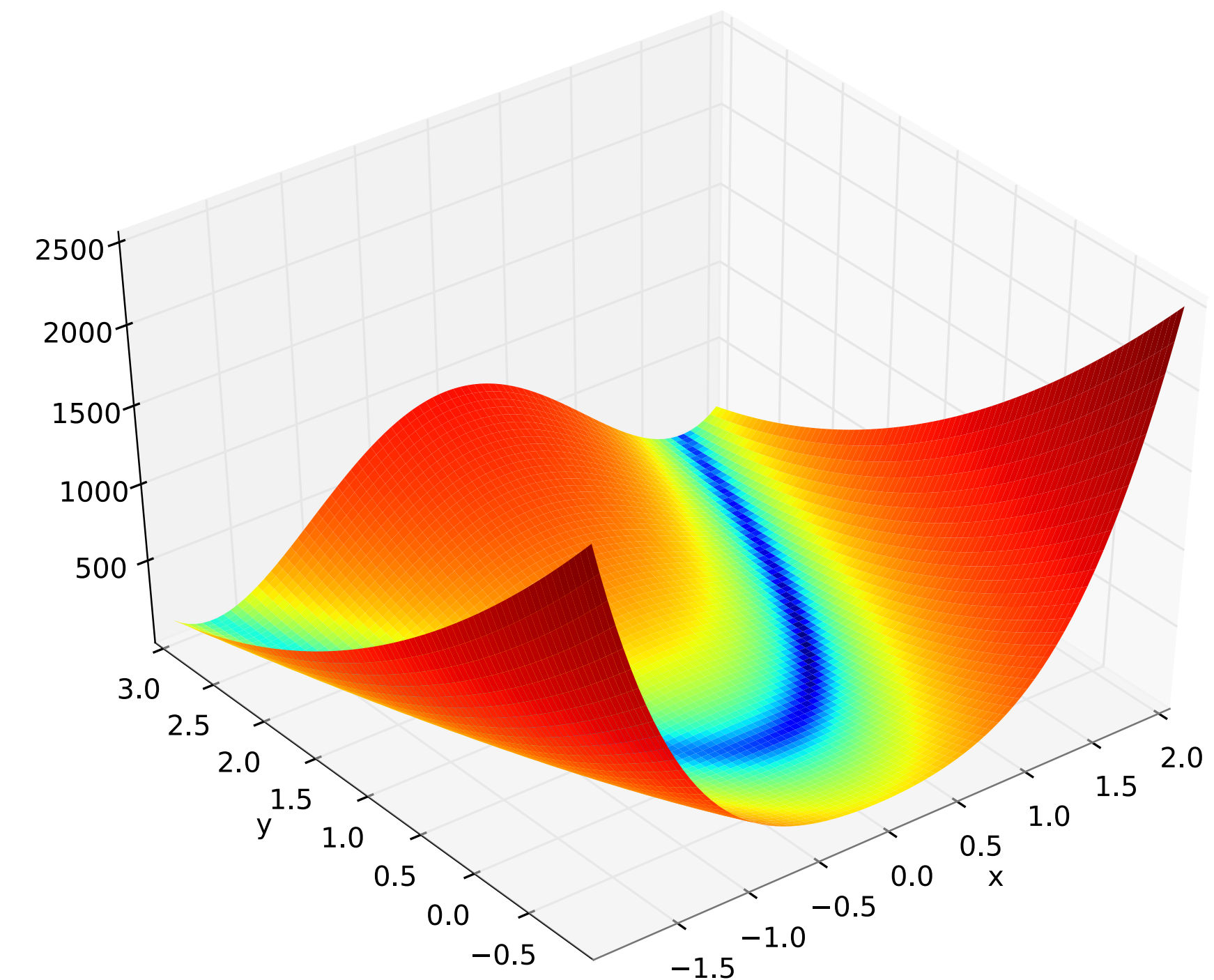
$$\overline{\mathbf{b}^{(1)}} = \bar{\mathbf{z}}$$

Stochastic gradient descent

Optimization method

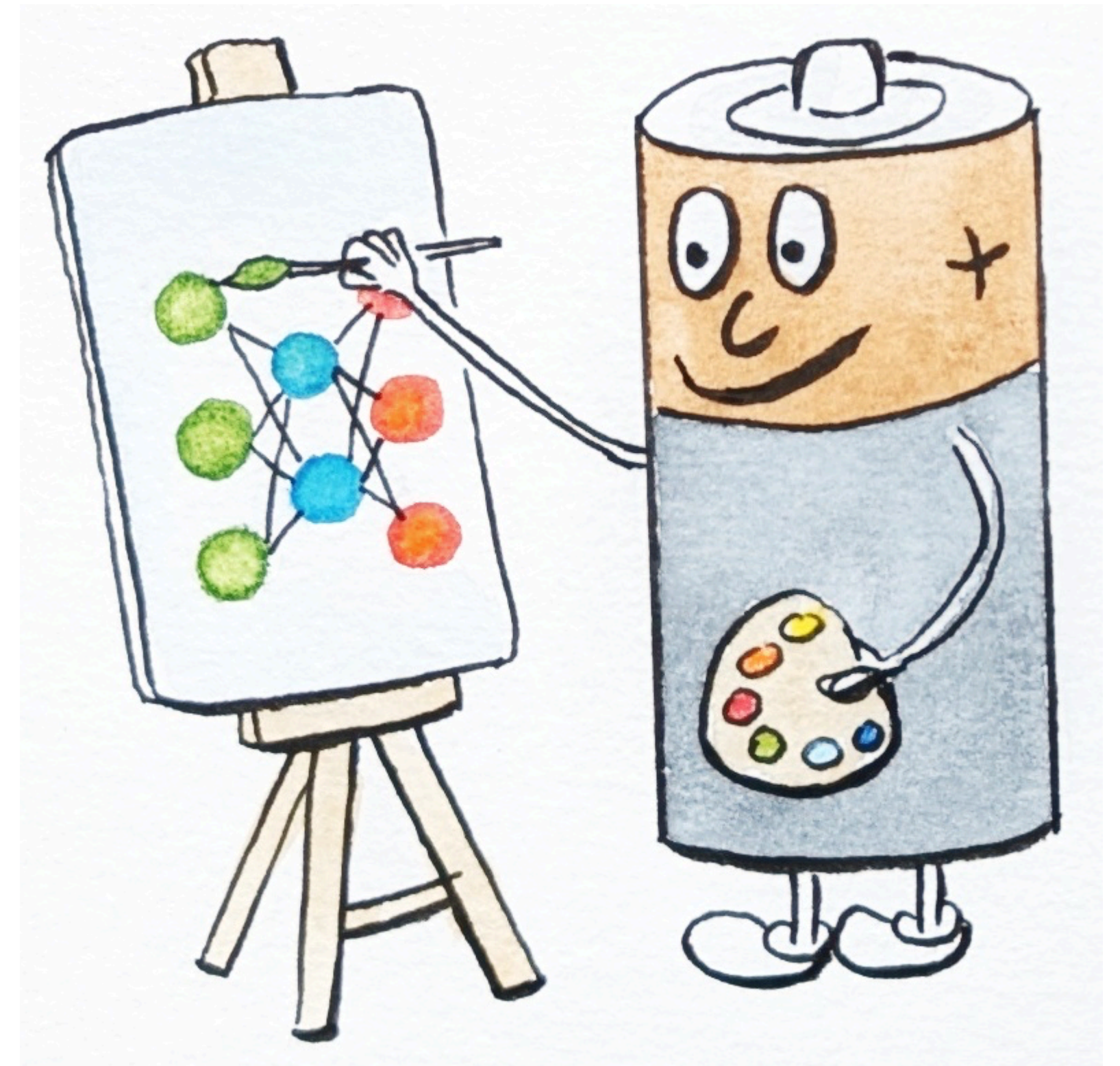
Gradient descent can write code better than you. I'm sorry.

Andrej Karpathy, 4.8.2017



https://commons.wikimedia.org/wiki/File:Rosenbrock_function.svg

Generative



(C) Adriana Harakalova, 2024

Input Text Processing

- **Tokenization** - a process in which a text is broken down into smaller units such as words, sentences, or characters, which are referred to as "tokens."
- **Encoding** - Assigning a unique numerical identifier to each token.
- small GPT-2 has approximately 50,257 tokens (**Byte Pair Encoding**).

Tokenization and Encoding

Character-based encoding example

H A L L O **Text**

Tokenization and Encoding

Character-based encoding example

H A L L O

Text

'H' 'A' 'L' 'L' 'O'

Tokens

Tokenization and Encoding

Character-based encoding example

H A L L O

Text

'H' 'A' 'L' 'L' 'O'

Tokens

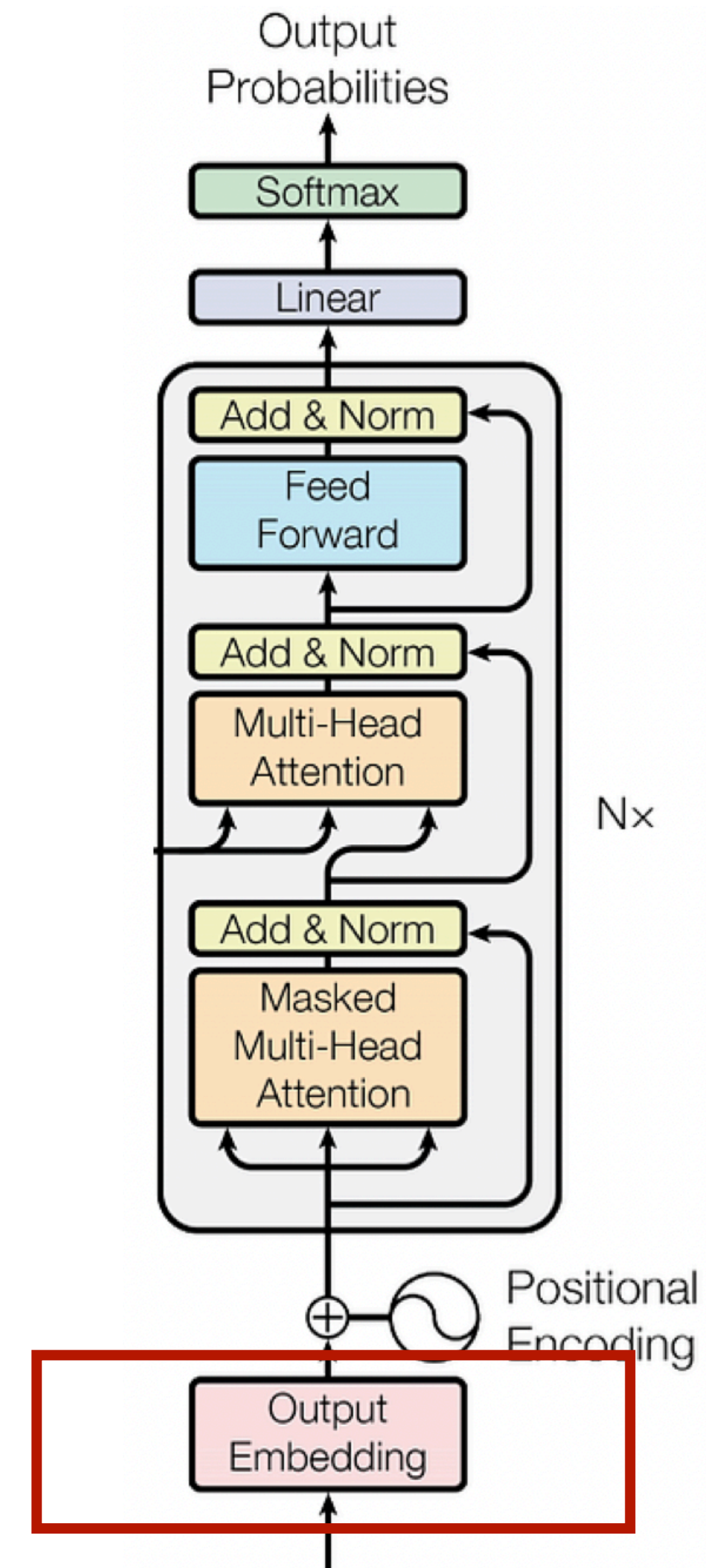
$\vec{v} = (8 \quad 1 \quad 12 \quad 12 \quad 15)$

Numerical
Sequence

Level 1

Embedding

- Every token processed in the **same time** - efficient
- every token represented as a vector in **multidimensional** space
- Values represent **semantic meaning**

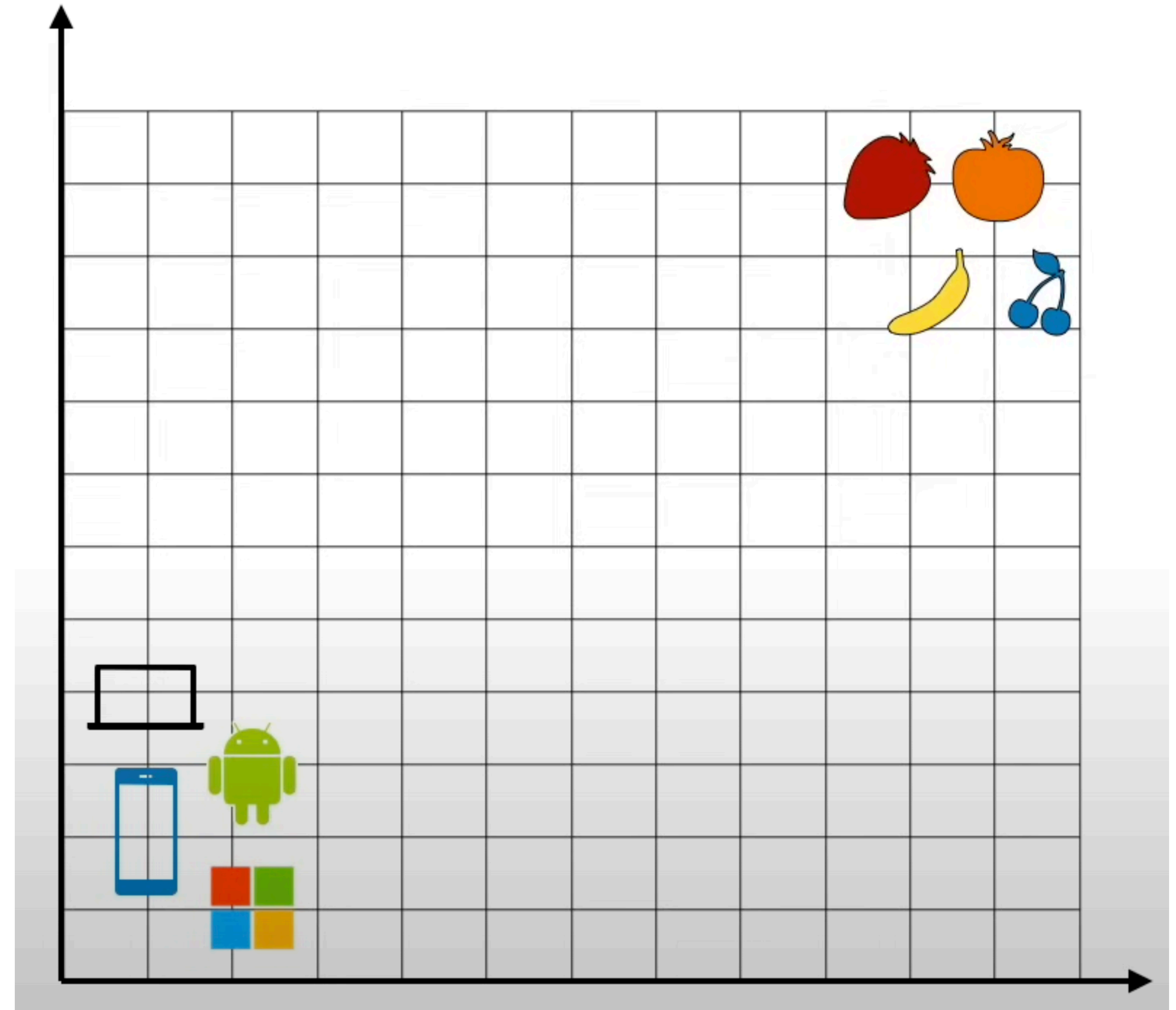


Michal Harakal, 2024/11

Embedding

Text as numbers
in multidimensional space

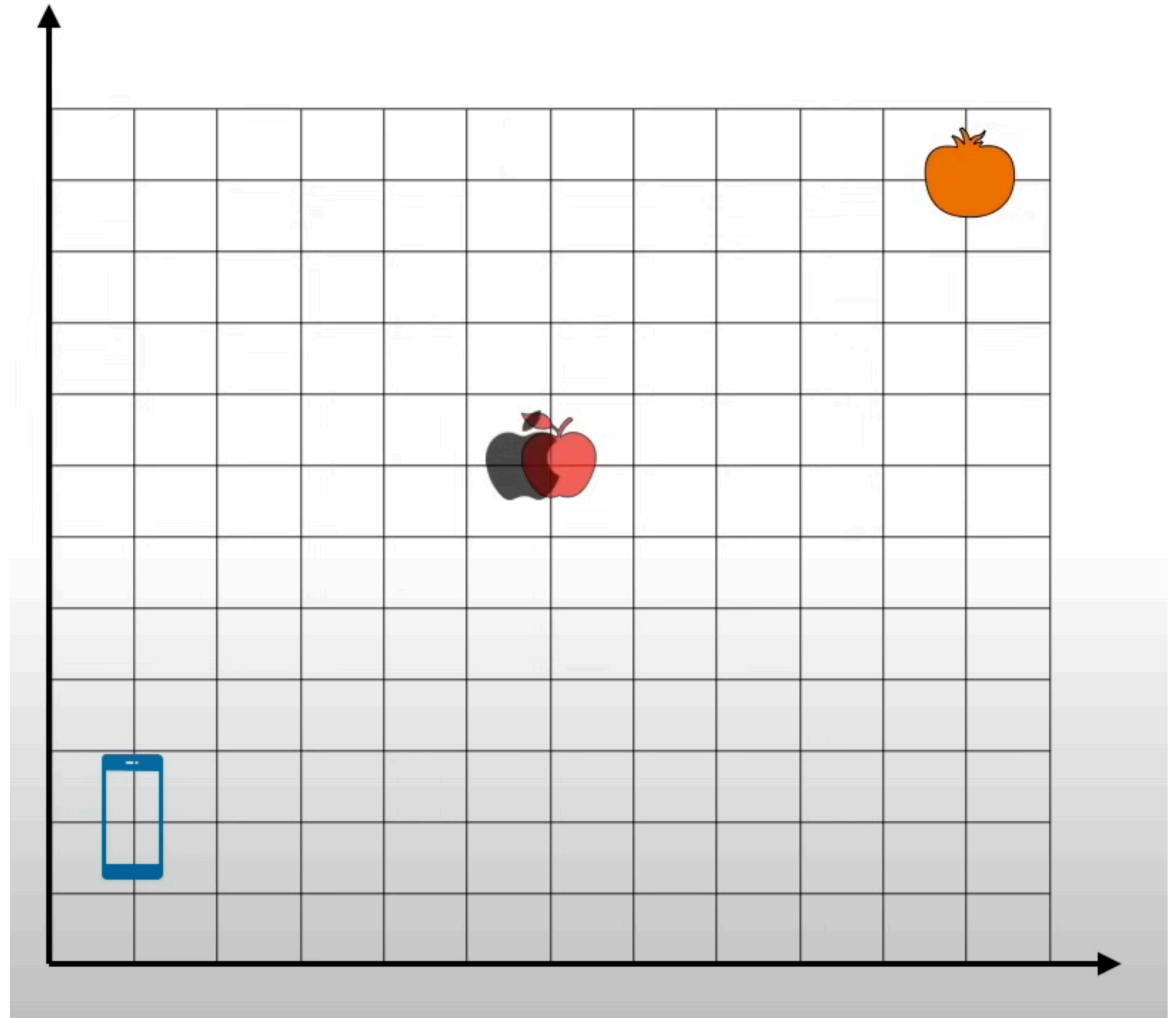
GTP2-768 dimensions



<https://www.youtube.com/watch?v=OxCpWwDCDFQ&list=PLs8w1Cdi-zvYskDS2iclltfZgxcIApVLv>

Positional encoding

- Please buy an apple and an orange
- Apple unveiled the new phone
- **CONTEXT**

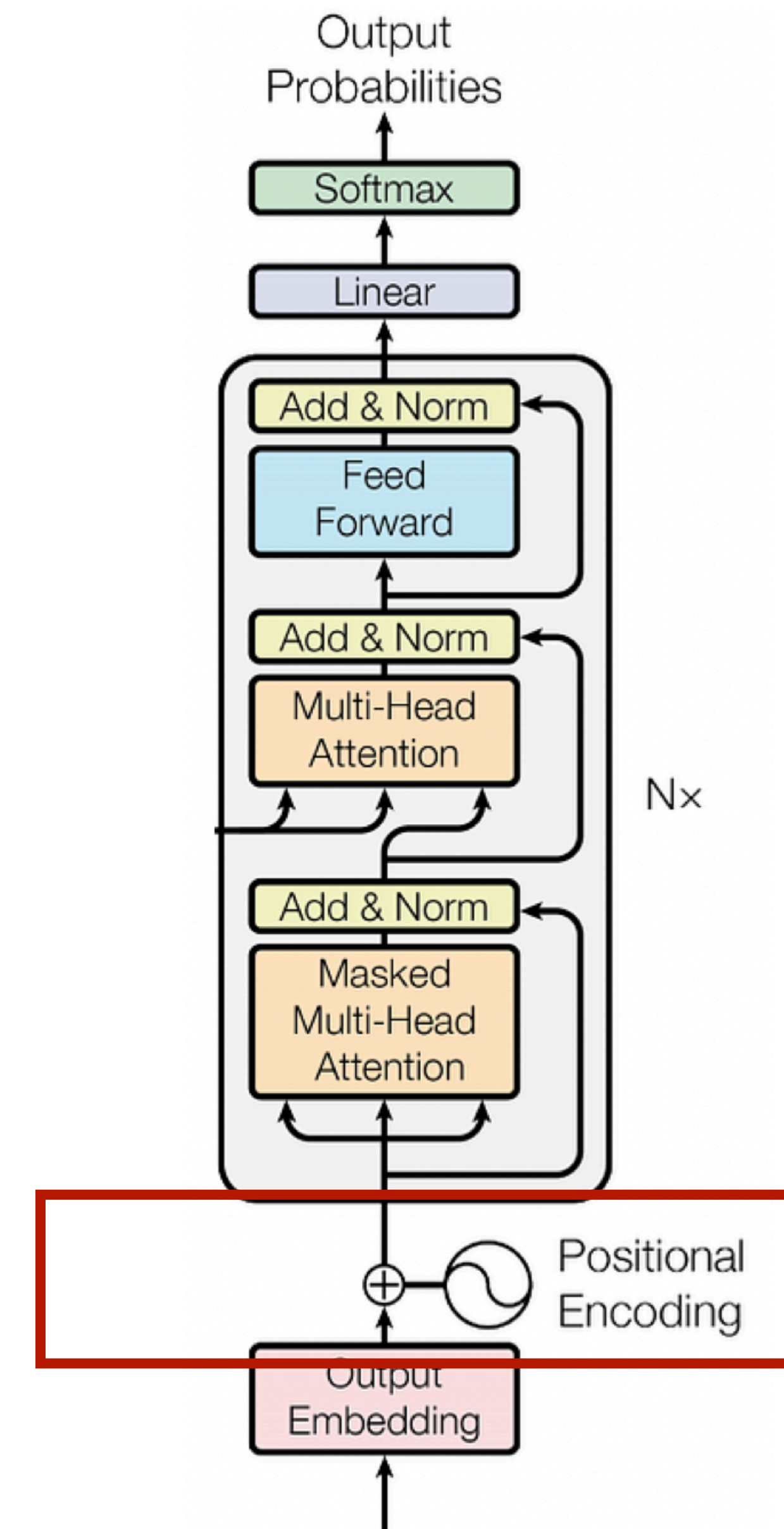


<https://www.youtube.com/watch?v=OxCpWwDCDFQ&list=PLs8w1Cdi-zvYskDS2iclltfZgxclApVLv>

Level 2

Positional Encoding

- provides information about the **order** in which words appear.
- Uses e.g sinusoidal functions (sine and cosine) to generate the positional encodings based on the position of the word in the sequence. This ensures that **words at different positions have different encodings**

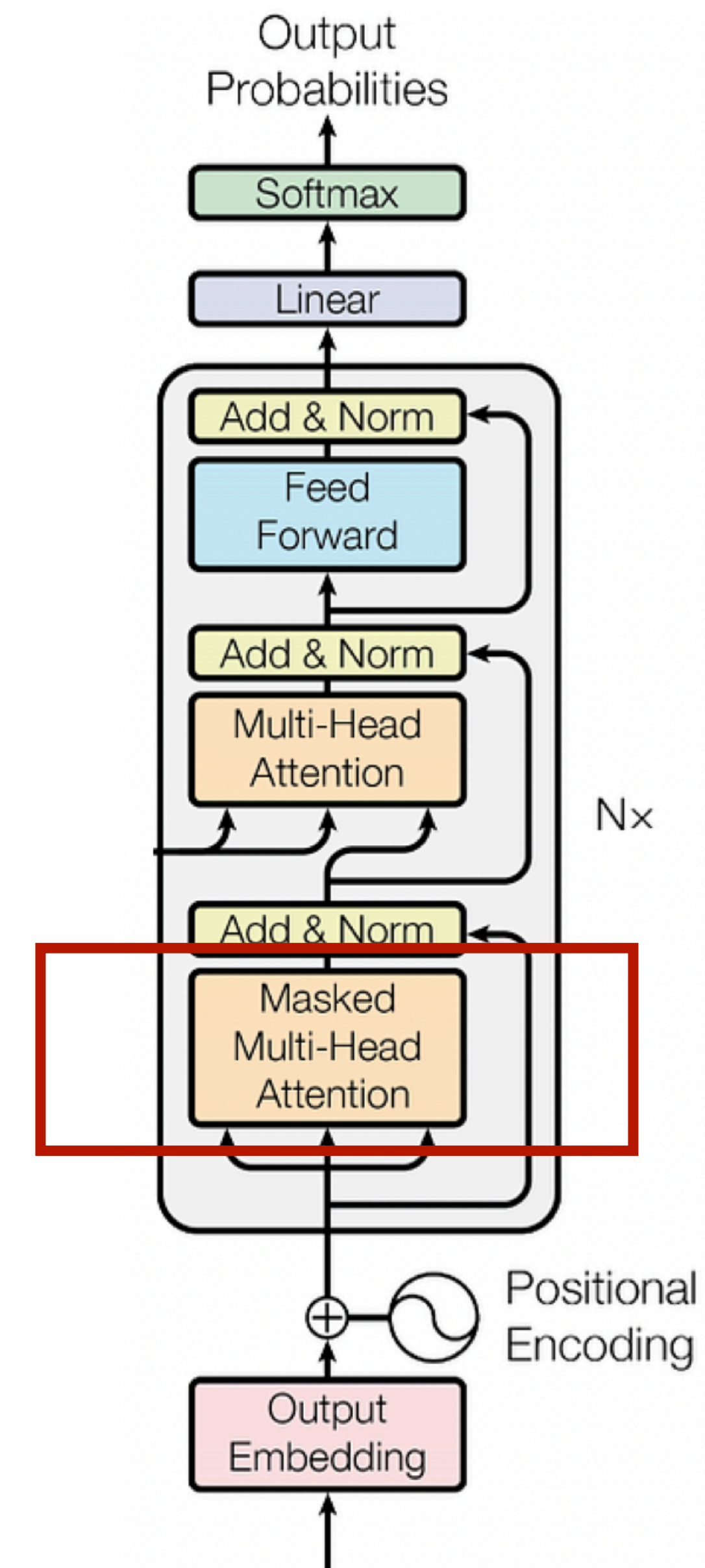


Michal Harakal, 2024/11

Level 3

Self-attention mechanism

- Context rich
- Attention captures contextual meaning

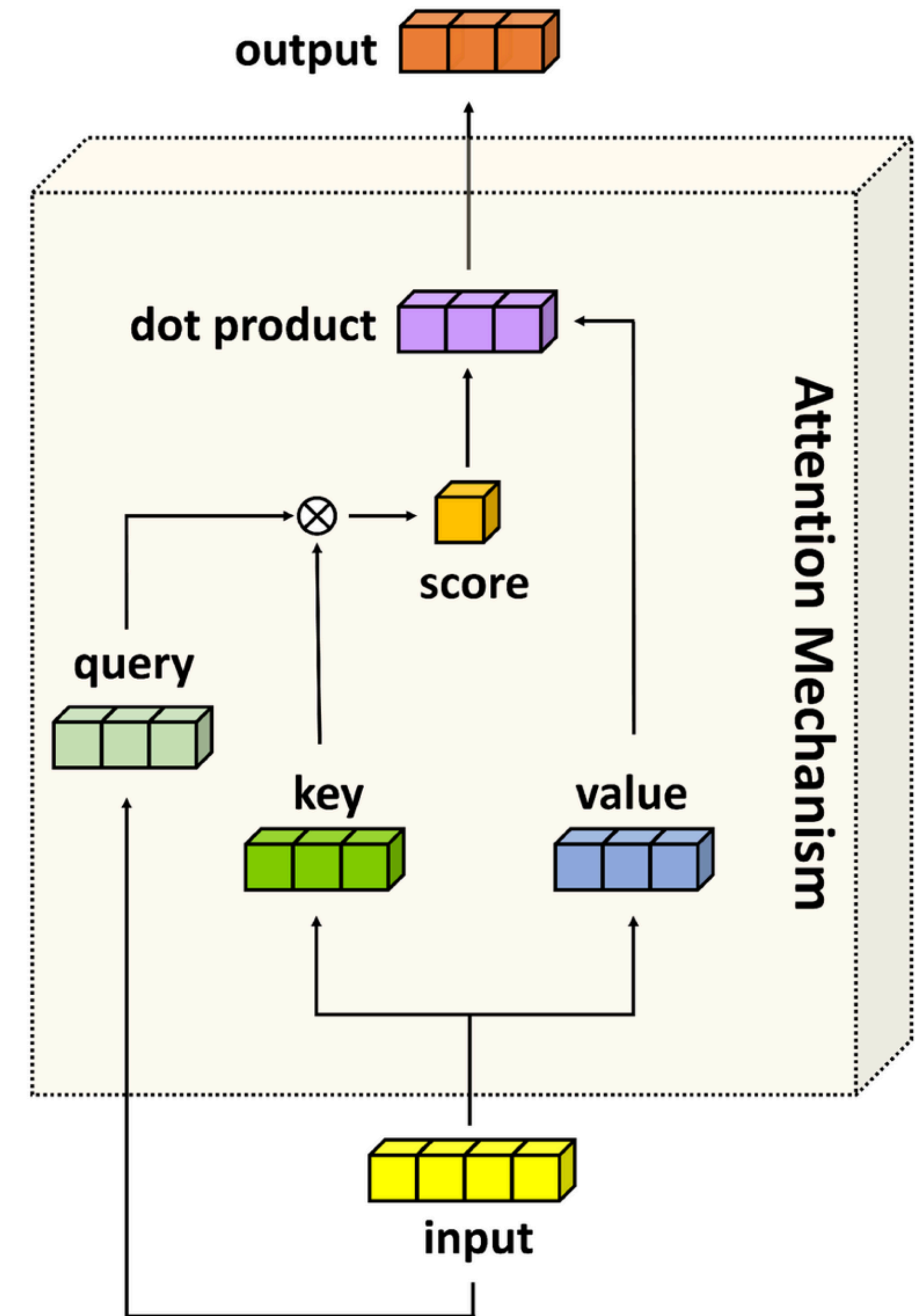


Michal Harakal, 2024/11

Level 3

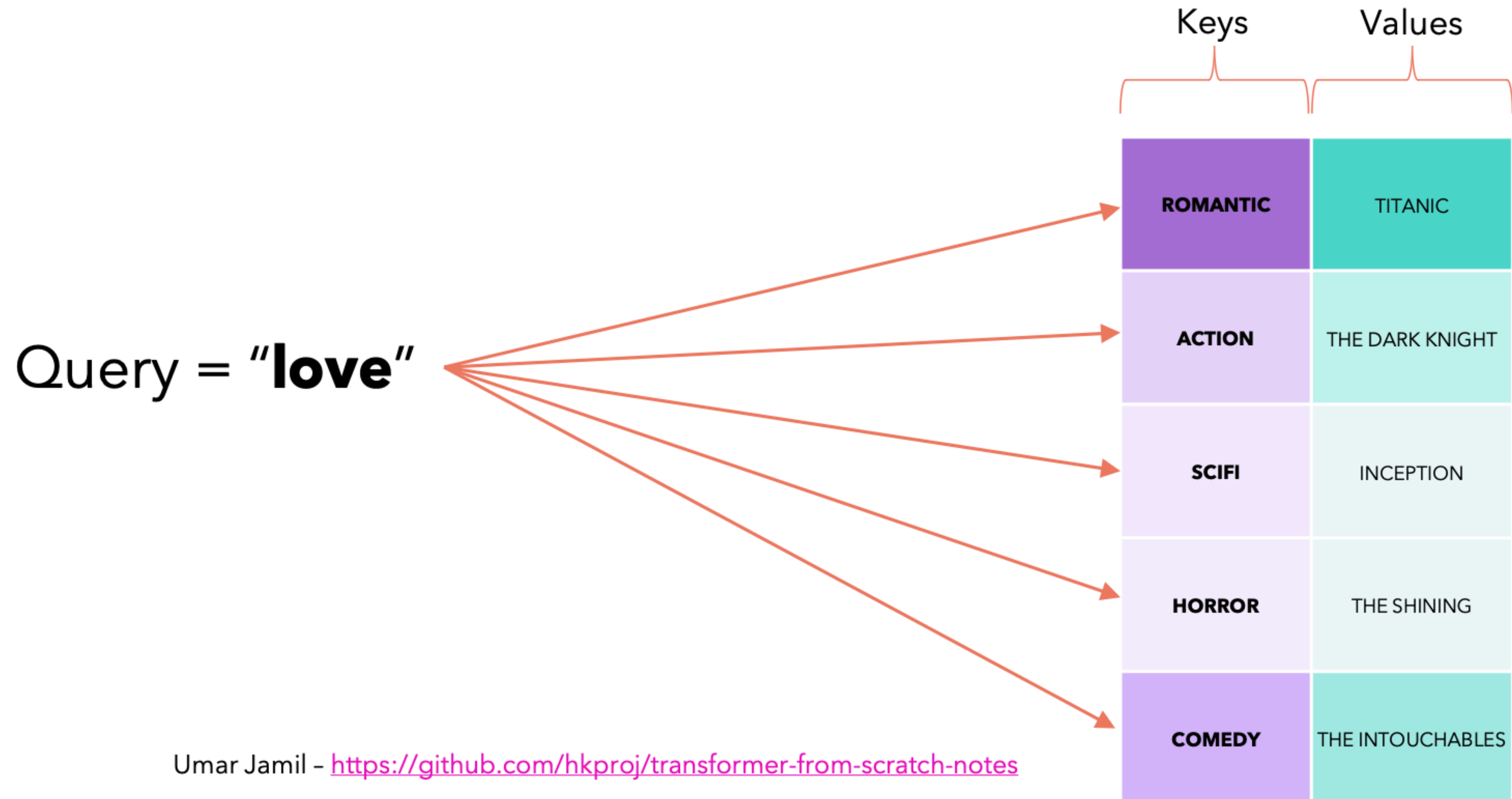
Self-attention mechanism

- **query (Q)** vector determines **what** the model is focusing on
- **key (K)** vector represents **potential matches** for the query
- **value (V)** holds information to be retrieved based on the attention score between the query and key



Query, keys and values

Can be HashTable or database table



Multihead is parallel

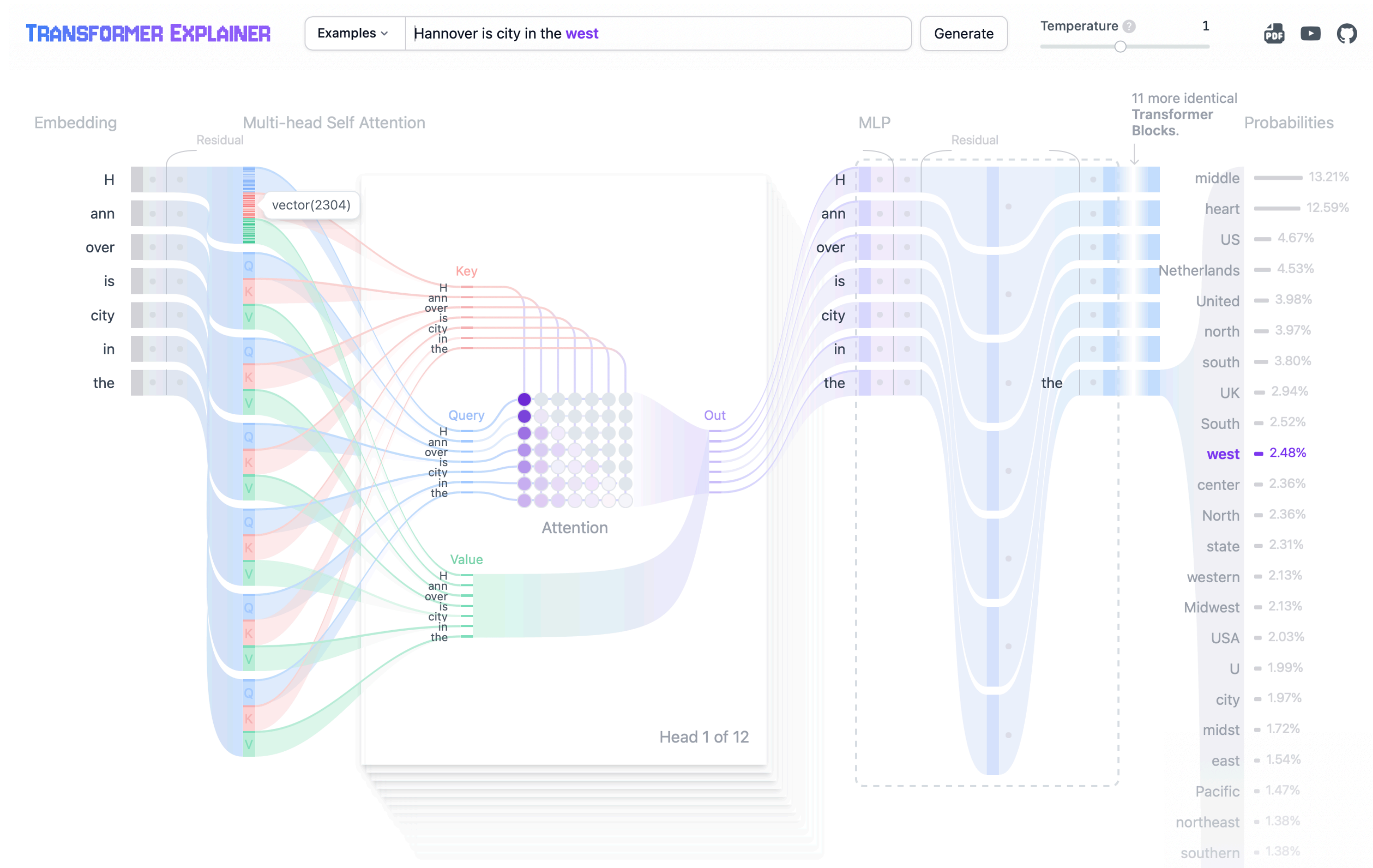
Kotlin coroutines

```
import kotlinx.coroutines.async
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.coroutineScope

override fun forward(input: Tensor): Tensor = runBlocking {
    // Run each head's forward method in parallel using coroutines
    val headOutputs = coroutineScope {
        _heads.map { head ->
            async {
                head.forward(input) // This will execute in parallel
            }
        }.map { it.await() } // Collect the results once all coroutines finish
    }
    ...
}
```

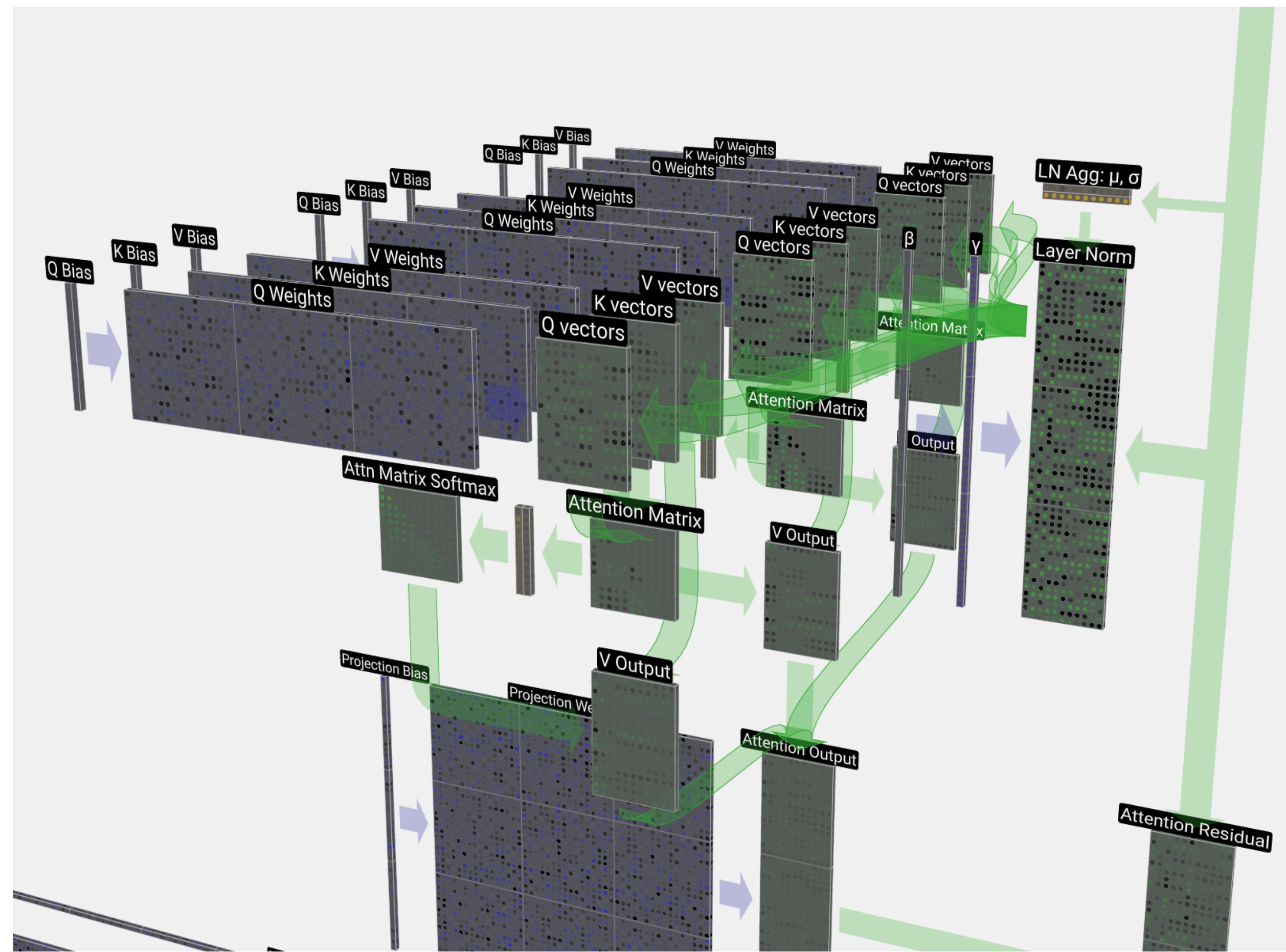
2D GTP-2 Visualization

<https://poloclub.github.io/transformer-explainer/>

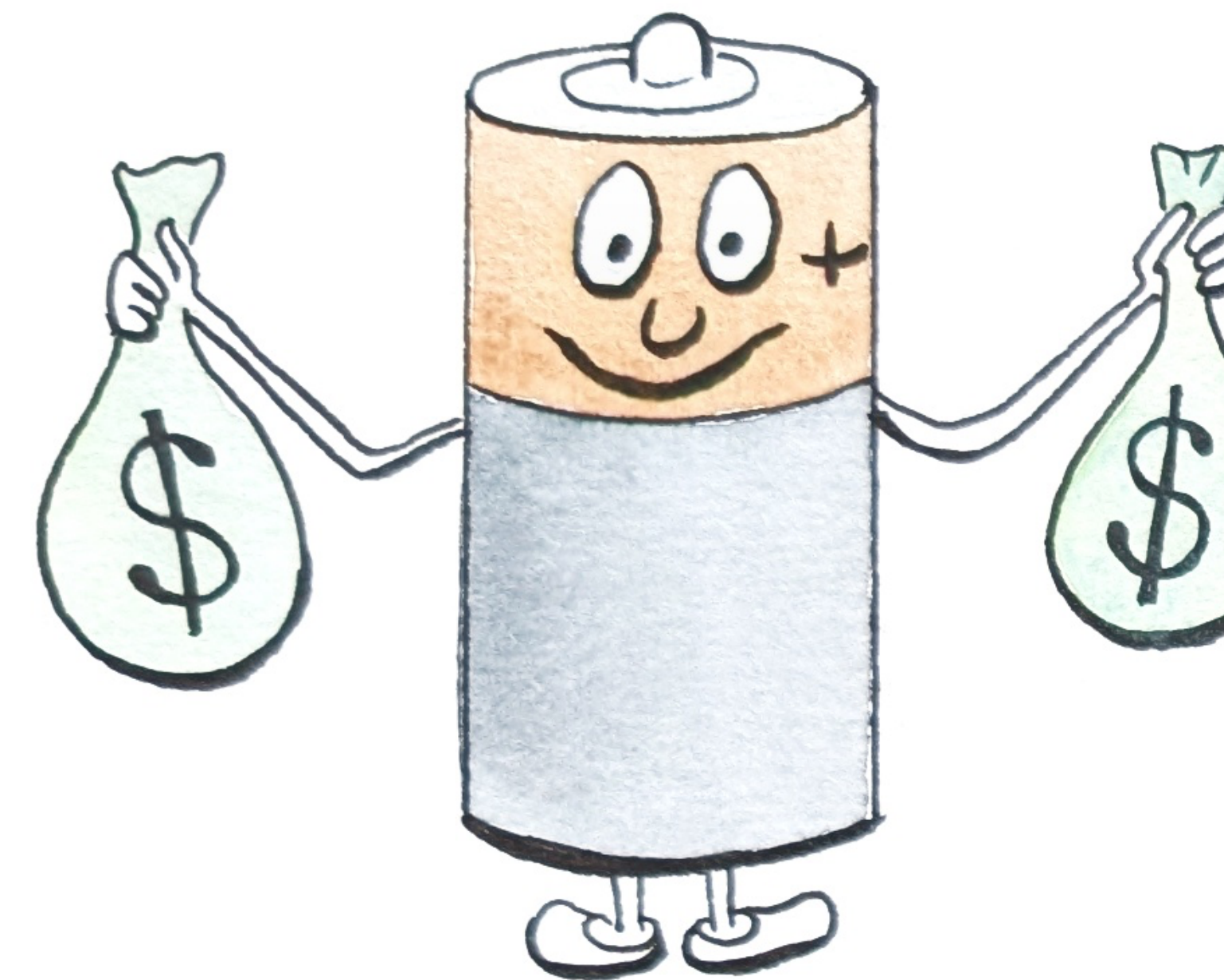


3D nanoGTP Visualization

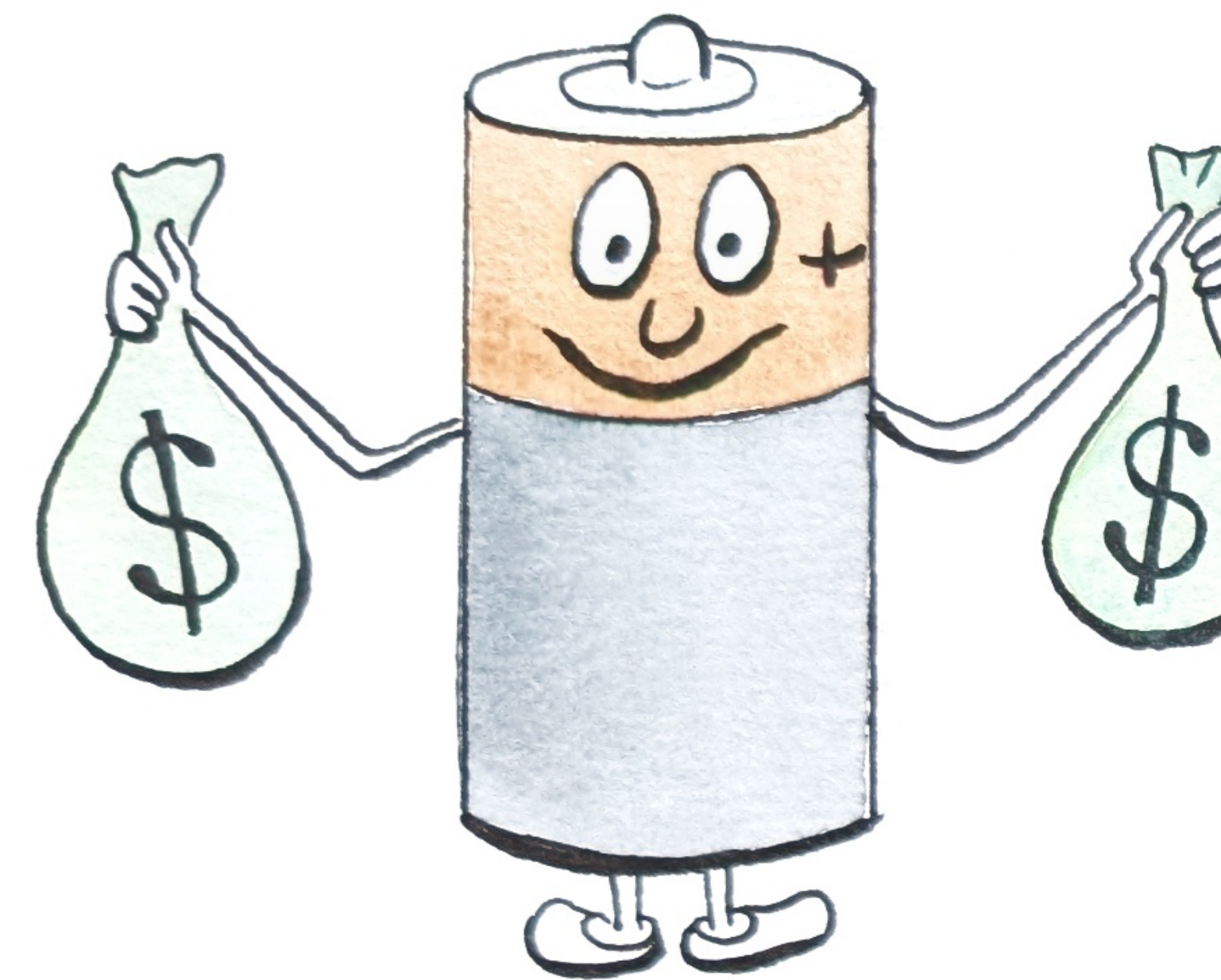
<https://bbycroft.net/llm>



Let's talk about ChatGPT



(C) Adriana Harakalova, 2024



(C) Adriana Harakalova, 2024

Let's talk about ChatGPT & again

Cloud AI is energy hungry

Amazon goes nuclear, acquires Cumulus Data's atomic datacenters for \$650M

E-commerce giant on the hook for 480MW of power from Susquehanna plant

Tobias Mann

Mon 4 Mar 2024 // 23:36 UTC

Amazon Web Services on Monday added a nuclear-powered datacenter campus to its public cloud empire as part of a \$650 million deal with Talen Energy – an owner and operator of electricity generation and transmission facilities in the US.

NATIONAL

Three Mile Island nuclear plant will reopen to power Microsoft data centers

SEPTEMBER 20, 2024 · 1:40 PM ET

By C Mandler



nuclear clean energy agreement Kairos Power

To accelerate the clean energy transition across the U.S., we’re signing the world’s first corporate agreement to purchase nuclear energy from multiple small modular reactors (SMR) to be developed by Kairos Power.

ell
tor, Energy and Climate

Share



Cloud AI issues

BLERP

- **Bandwidth** - As an example, a device at 100Hz sampling rate produces 360,000 data points each hour. Now imagine the amount of data produced by a fleet of these devices. It get's even trickier with images and video.
- **Latency** - "time between when a system takes in a sensory input and responds to it". In case of conventional ML deployment data must be first sent to an ML application. This increases the time in which an edge device can take action as it waits for the response.
- **Economics** - Cloud is cheap but not so cheap. It still costs money to ingest large amounts of data, especially if it must happen in real-time.
- **Reliability** - Revisiting the bandwidth example, in case of high-frequency sampling, it might be hard to ensure that data arrives to a target in the same order as it was produced by an edge device.
- **Privacy** - processing data on-device and is not sent through network. This reduces the surface for data abuse.

Cloud AI issues

- **Centralized** knowledge => power
- **Close sourced**
- **Data privacy concerns**

On-device AI



On-device AI

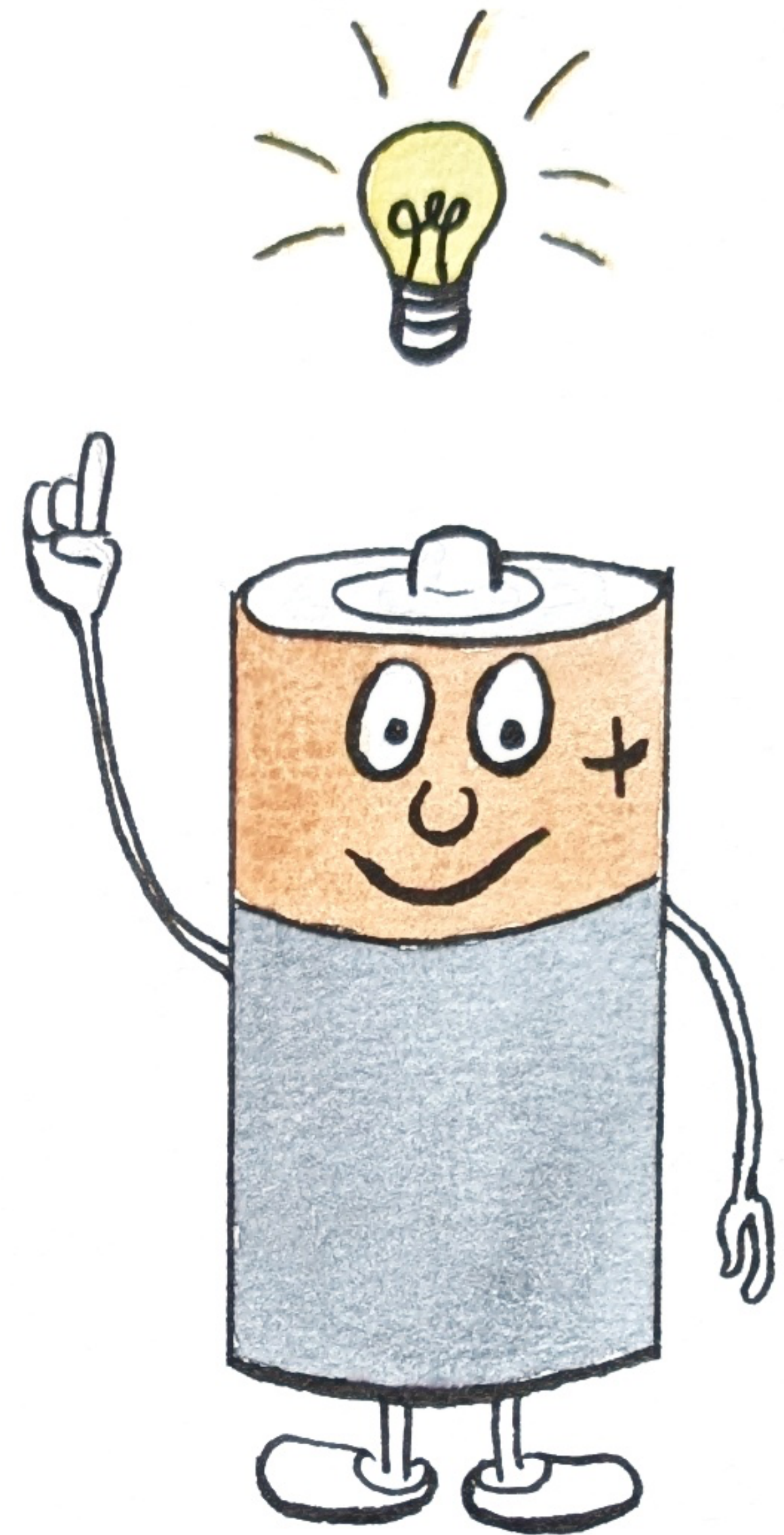
Local Processing: Executes AI computations directly on the device (like a smartphone, tablet, IoT device, or embedded system) rather than relying on cloud servers.

On-device AI

Advantages

- **Privacy-First**- no data sharing
- **Low Latency** - ideal for real-time applications (e.g., voice assistants, face recognition, augmented reality).
- **Offline Capability**- allowing uninterrupted functionality in areas with limited or no connectivity.
- **Optimized for Limited Resources**: Uses smaller, highly efficient models adapted to run on devices with limited processing power, memory, and battery life.
- **Cost-Effective**: Reduces dependency on cloud resources, lowering operational costs related to data transfer and processing.
- **Enhanced User Experience** - Provides seamless and instant responses in applications, improving interactivity and user satisfaction.

sKaiNET



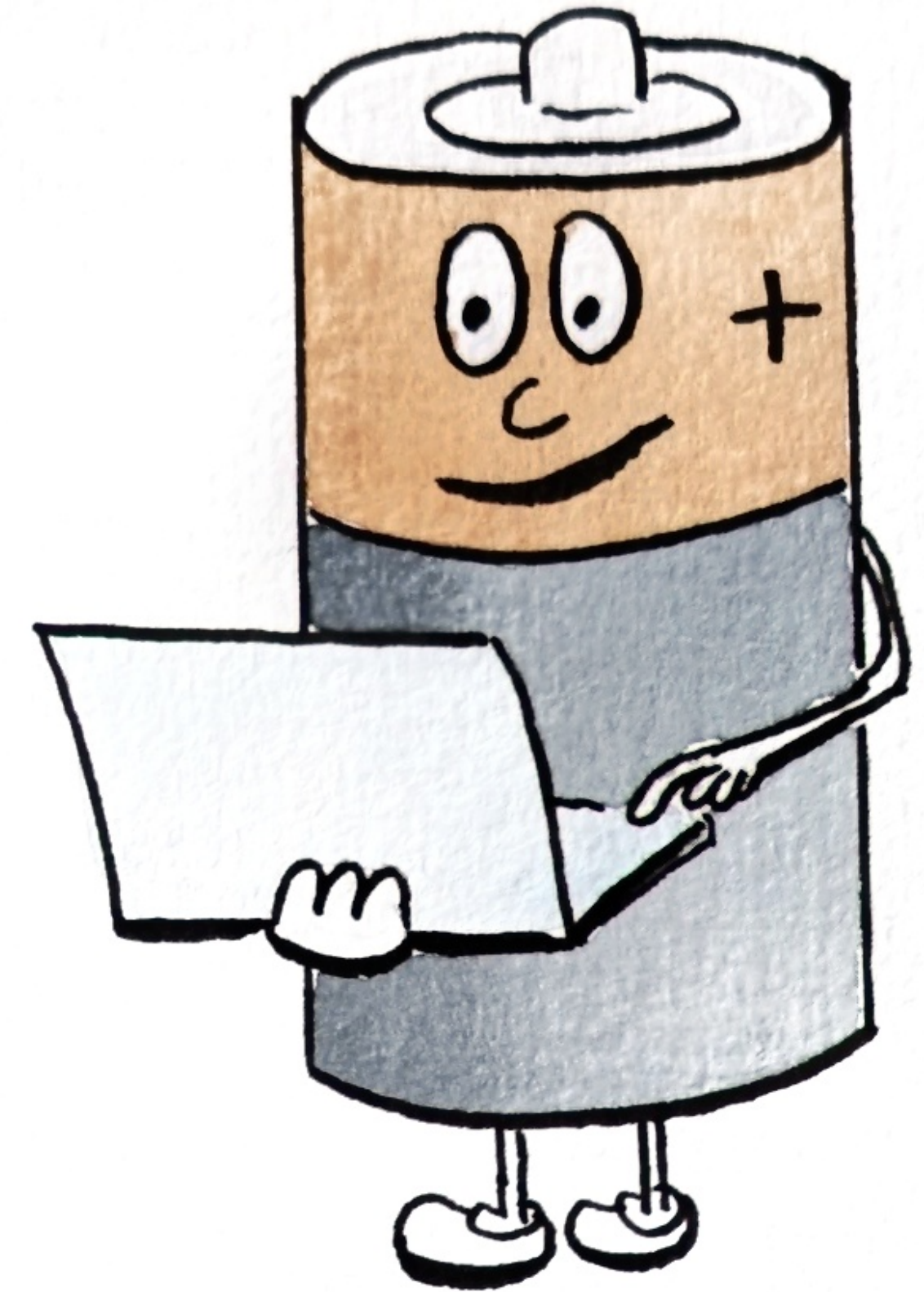
(C) Adriana Harakalova, 2024

Building on-device first ML framework

sKaiNET

- Focus on-device first
 - Offline capability
 - Constrained resources (memory, cpu ...)
 - Connectivity
- Developer oriented
- Simple

Software development meets ML



(C) Adriana Harakalova, 2024

Key ideas

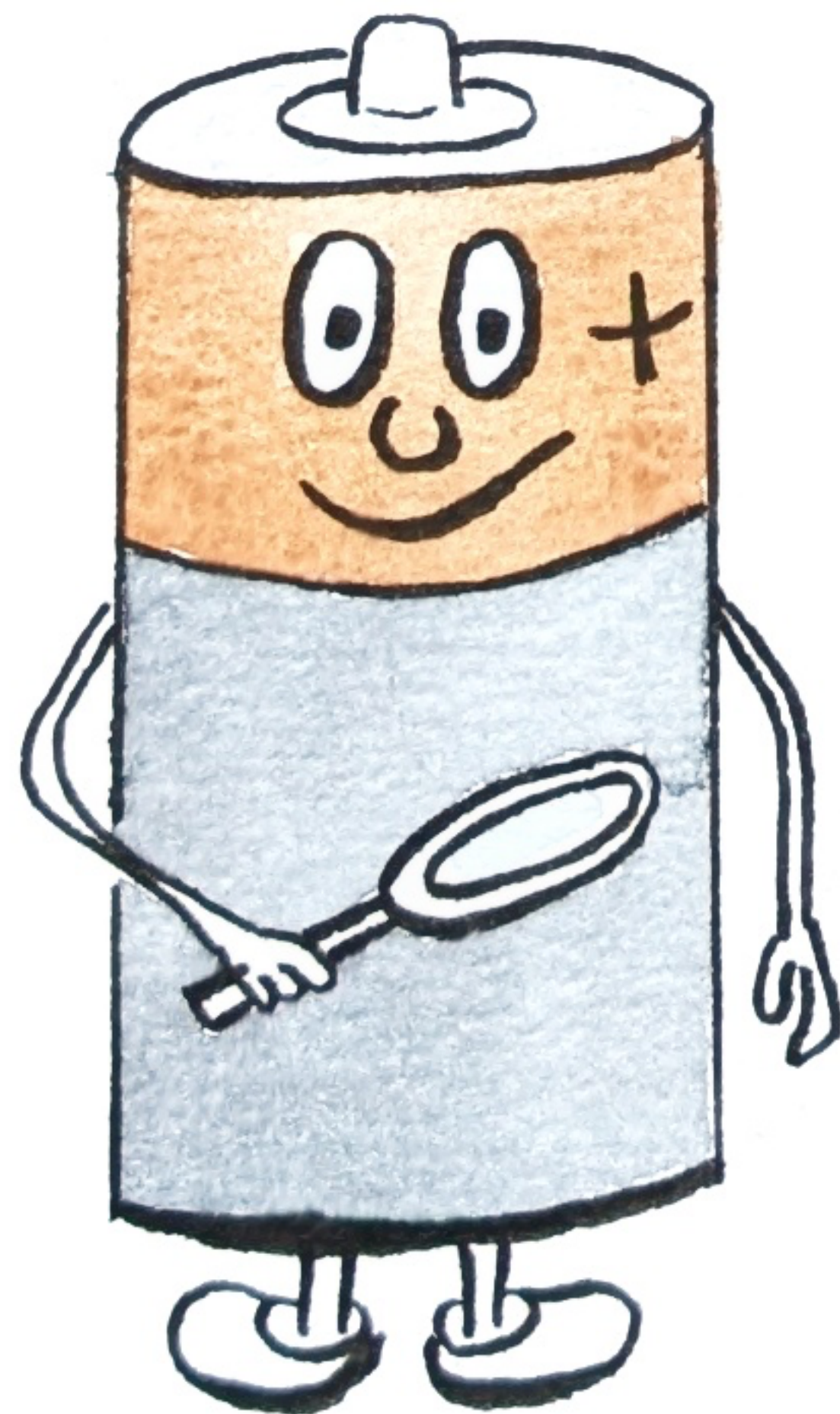
Let's learn from software development practices

- Use of dedicated DSLs for model definition, tensors slicing or visualization
- Modularization
- Separation of Training, Evaluation & Inference
- Compiled LLMs

Compiled ML

In Kotlin, inspired by K2

- **Frontend** - Model definition as DSLs
- **Backend**
 - Conversion into compute graph via Kotlin compiler Plugin,
 - Conversion into intermediate representation
 - Optimization on code level
 - Optimizations on compute graph operators level
- **Linker** Use existing LLVM infrastructure for compiling into native platform



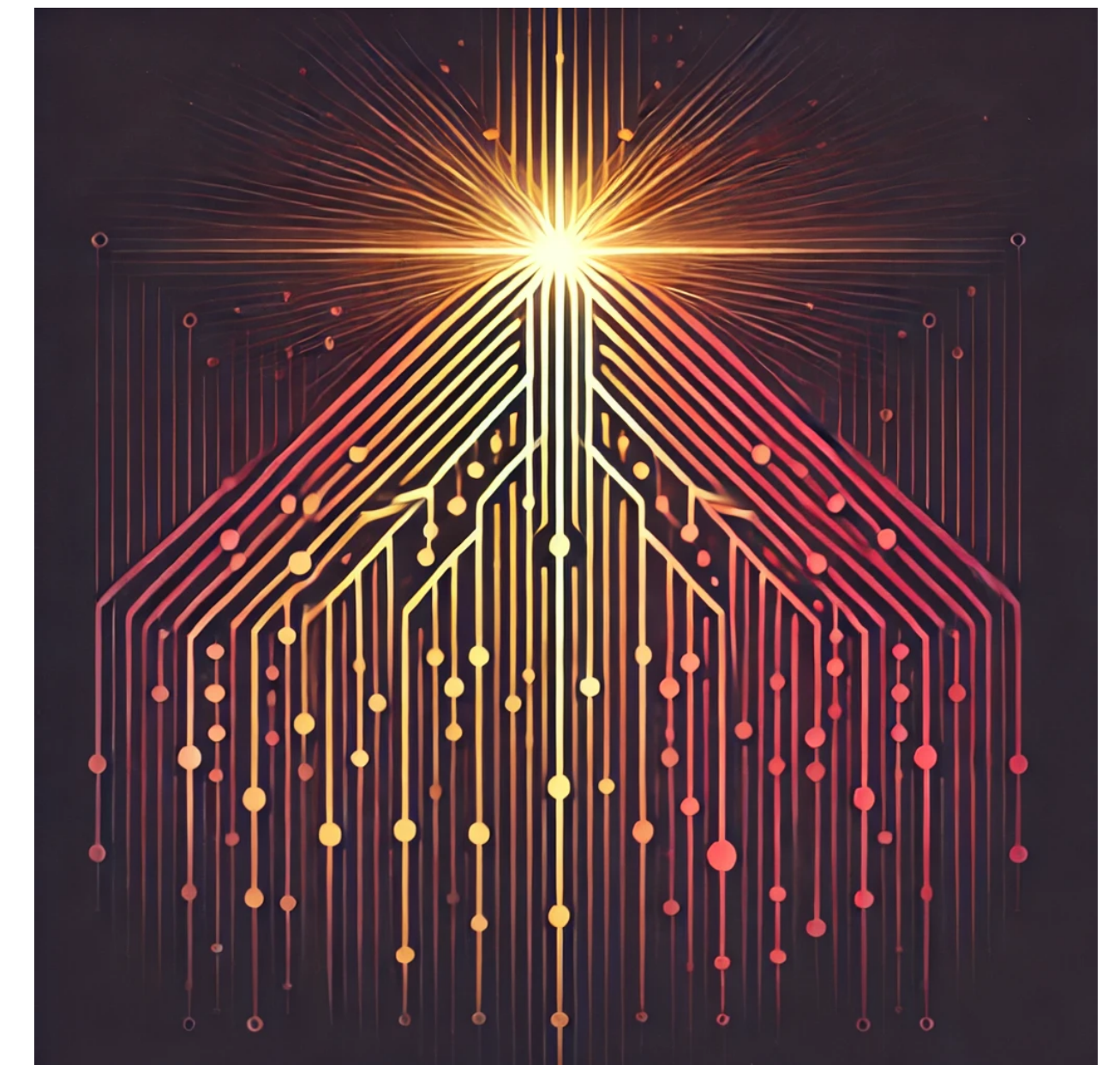
(C) Adriana Harakalova, 2024

Testing

gradienttracer

Testing like a boss

```
your_data/  
├── core.py  
├── TS-001/  
│   ├── UC-001.py  
│   └── UC-002.py  
├── TS-002/  
│   ├── UC-001.py  
│   └── UC-002.py  
└── TS-003/  
    ├── UC-001.py  
    └── UC-002.py
```



```
from core import Executable
import torch

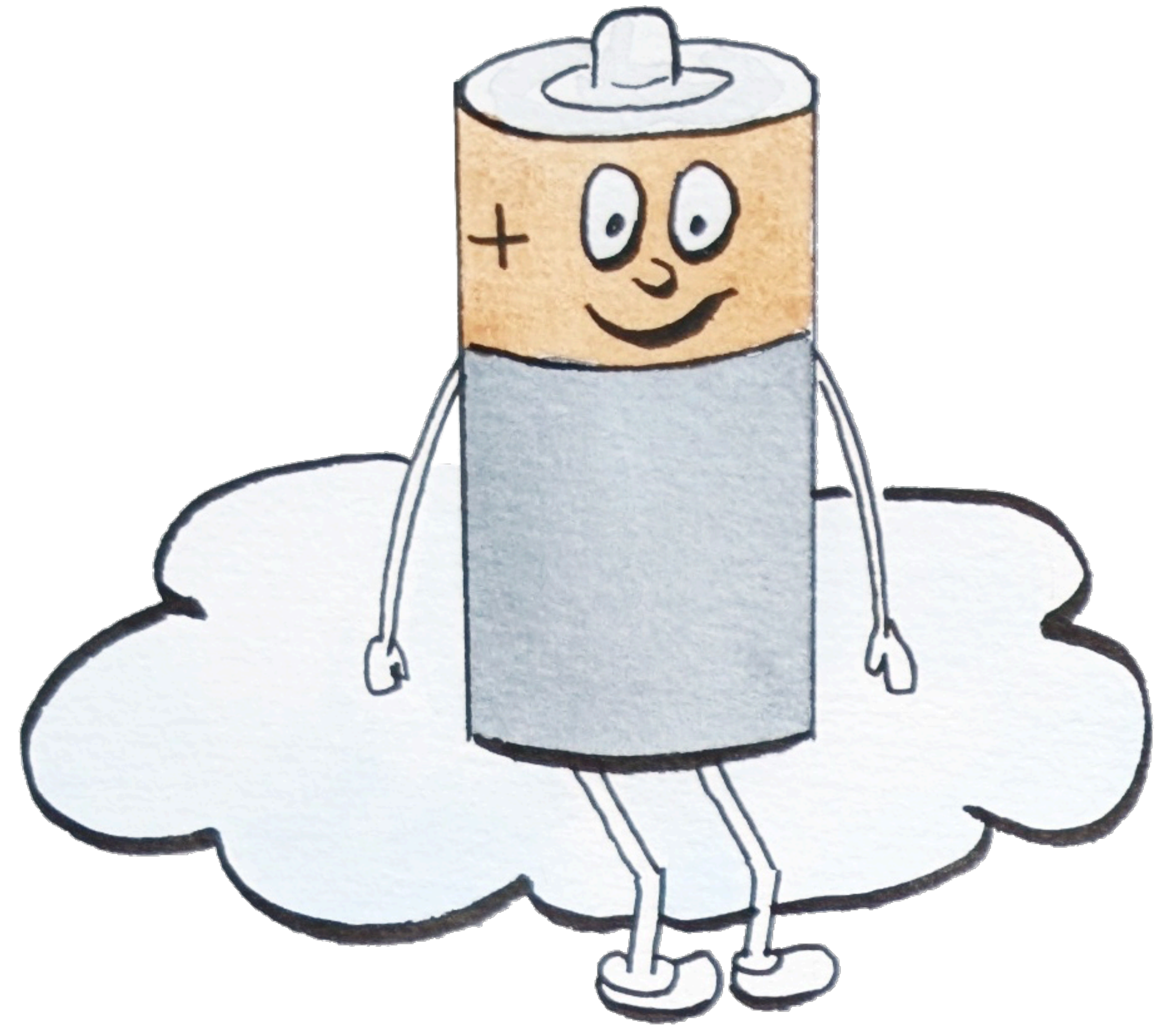
@Executable('simple expression with relu')
def simple_expression_with_relu():
    x = torch.Tensor([-4.0]).double()
    x.requires_grad = True
    z = 2 * x + 2 + x
    q = z.relu() + z * x
    h = (z * z).relu()
    y = h + q + q * x
    y.backward()
    return [x], y
```

ML in Kotlin from scratch

Kotlin libraries

- Standing on the shoulders of giants
 - Kotlin port of nanoGPT -> 80%
 - Now as Kotlin Multiplatform
 - <https://github.com/michalharakal/KPTChat/tree/feature/kmp>
- Kotlin port of micrograd -> 90%
 - With its own DSL language for graphviz DOT language
 - <https://github.com/michalharakal/miKrograd>
- Testing framework for validation with Pytorch calculations
 - <https://github.com/michalharakal/gradienttracer>

Demo



(C) Adriana Harakalova, 2024

Thank you



<https://github.com/michalharakal/from-zero-to-kotlin-gpt/tree/KTUG-2024>



Links

Sources and links

- Attention Is All You Need - <https://arxiv.org/pdf/1706.03762>
- Let's build GPT: from scratch, in code, spelled out (<https://www.youtube.com/watch?v=kCc8FmEb1nY>)
- Pytorch logo https://commons.wikimedia.org/wiki/File:Pytorch_logo.png
- <https://www.neuromedia.ca/how-many-neurons-are-in-the-human-brain/>
- The math behind Attention: Keys, Queries, and Values matrices
https://www.youtube.com/watch?v=UPtG_38Oq8o&list=PLs8w1Cdi-zvYskDS2icIltfZgxclApVLv&index=2
- https://commons.wikimedia.org/wiki/File:Rosenbrock_function.svg <https://github.com/michalharakal/KPTChat/tree/feature/kmp>
- <https://github.com/michalharakal/miKrograd>
- <https://lmos-ai.github.io/arc/>
- <https://paperswithcode.com/paper/kan-kolmogorov-arnold-networks>
- <https://github.com/kindxiaoming/pykan>
- <https://commons.wikimedia.org/wiki/File:The-Transformer-model-architecture.png>
- <https://github.com/michalharakal/gradienttracer>
- <https://deeplearning4j.konduit.ai/>

Links

Sources and links

- <https://github.com/hkproj/transformer-from-scratch-notes>

Links

Sources and links

- Attention Is All You Need - <https://arxiv.org/pdf/1706.03762>
- Let's build GPT: from scratch, in code, spelled out (<https://www.youtube.com/watch?v=kCc8FmEb1nY>)
- Pytorch logo https://commons.wikimedia.org/wiki/File:Pytorch_logo.png
- <https://www.neuromedia.ca/how-many-neurons-are-in-the-human-brain/>
- The math behind Attention: Keys, Queries, and Values matrices
https://www.youtube.com/watch?v=UPtG_38Oq8o&list=PLs8w1Cdi-zvYskDS2icIltfZgxclApVLv&index=2
- https://commons.wikimedia.org/wiki/File:Rosenbrock_function.svg <https://github.com/michalharakal/KPTChat/tree/feature/kmp>
- <https://github.com/michalharakal/miKrograd>
- <https://lmos-ai.github.io/arc/>
- <https://paperswithcode.com/paper/kan-kolmogorov-arnold-networks>
- <https://github.com/kindxiaoming/pykan>
- <https://commons.wikimedia.org/wiki/File:The-Transformer-model-architecture.png>
- <https://github.com/michalharakal/gradienttracer>
- <https://deeplearning4j.konduit.ai/>